

**R-Figure 1: A motivation example of our path-based semantic subgraph matching method. As our method is path-aware, it is designed to address the inflexible graph matching issue, where the query graph and candidate data graph can be semantically equal but not topologically the same.**

We sincerely thank the efforts of all reviewers on reviewing our submission. For clarification, we have summarized reviews from reviewers and addressed them. The revisions to address reviews are highlighted by the blue color in the revised submission. For easy checking, we have included Figures and Tables in the response letter, labeled as R-Figure and R-Table, respectively. Our detailed replies are as follows:

## 1 META-REVIEWER

**Meta-R1:** Clear Repositioning Toward Query Retrieval and Data Management: The paper must be reframed to emphasize query retrieval and graph data management as its central contribution, rather than primarily an LLM/RAG engineering pipeline.

What is required

- Revise the title, introduction, and related work to foreground query retrieval over graphs and database-style problem formulations.
- Include at least one concrete query-retrieval-oriented motivating example early in the paper.
- Adjust the narrative to highlight data-management challenges (graph construction, matching, efficiency, tradeoffs).

How this will be evaluated

- The introduction and related work clearly situate the paper among query retrieval, graph databases, and data management literature.
- The core contributions are framed as solving well-defined data/graph problems, with LLMs positioned as a context rather than the main contribution.

**Reply to Meta-R1:** Thanks for your insightful suggestions. We have revised the following of our paper according to your suggestions:

- We have revised the title of our paper to “MGRAG: Semantic Subgraph Matching and Graph-Aware Caching for Multimodal Retrieval-Augmented Generation”, which highlights the query retrieval aspect of our paper. Moreover, we also modified the Introduction (Section 1) and Related work (Section 2) part of our paper to further emphasize the query retrieval and data management aspect of our paper, and clearly position our contributions (Semantic path-aware) over query retrieval.

- In the new Introduction (Section 1) of our revised paper, we add a concrete motivating example in R-Figure 1 (Figure 1 here in revision plan) to further illustrate the query retrieval scenario where the query graph and candidate data graph are semantically equal but not topologically the same (Namely **inflexible graph matching** issue), which motivates our semantic path-based subgraph matching approach. We also add the related discussion in the Introduction (Section 1) of our revised paper.
- In our revised Introduction (Section 1) and Related work (Section 2), we frame our core contributions as solving the path-based semantic subgraph matching problem defined in Definition 2. Moreover, we further emphasize the current data management challenges of query retrieval and efficiency over graphs, they are: (1). **Inflexible graph matching**: The current subgraph matching method fails to handle the potential matching when the query graph and candidate data graph are semantically equal but not topologically the same. (2). **Event-agnostic KV re-computation**: In graph-based LLM data-management scenarios, the current KV cache mechanism fails to consider the tokens’ graph topological importance on the graph. These two issues are the key challenges for query retrieval and data management over graphs under LLM contexts, and our contributions are designed to address these two issues. We have added the related discussion in the Introduction (Section 1) and Related work part (Section 2) of our revised paper.

**Meta-R2:** Strengthened Related Work and Novelty Positioning: The paper must engage seriously with prior work in graph construction, subgraph matching, and graph/query retrieval, and clarify novelty relative to that literature.

What is required

- Discuss existing knowledge graph construction methods and subgraph/isomorphism-based matching approaches that could be adapted to similar tasks.
- Explicitly compare the proposed contributions (especially the first two) against these lines of work, clarifying what is new and what is adapted.

How this will be evaluated

- The related work section demonstrates awareness of relevant graph mining and graph matching literature.
- The novelty of each claimed contribution is clearly articulated in contrast to prior methods, not only to RAG-style systems.

**Reply to Meta-R2:** Thanks for your insightful suggestions. In our revised paper, we have revised the following of our paper according to your suggestions:

- In the revised related work part (Section 2), we further discussed the existing knowledge graph construction methods. For subgraph/isomorphism-based matching approaches, in the related work part, we further discussed the exact solver (VF2 [13], ISMAGS [28]), approximate match algorithms (SM [35], VJ [17], RRWM [11], IPFP [36]), and semantic match algorithms (BGE [58], SLOAlign [52], FGWAlign [53]). Although the KG construction approach is not part of our novelty contribution, we still compare our LLM-based KG construction method with the

existing knowledge graph construction methods, including Stanford OpenIE (O.IE) [6] and STAT [57]. The introduction of these KG construction baselines are listed in Section 4.1.3 and the result is listed in R-Table 2 (Table 5). For all these baselines, we also add the related experiment comparing them with our LLM-based KG construction and path-based semantic subgraph matching method in Section 4.6 of our revised paper, and the result is listed in R-Table 2 (Table 5). According to the experiment result, our LLM-based KG construction method and path-based semantic subgraph matching method both achieves better performance than other baselines, demonstrating the effectiveness of our proposed methods. We have added the experiment related discussion in Section 4.4 of our revised paper.

- Compared to previous subgraph matching methods, our path-based semantic subgraph matching method can handle the **in-flexible graph matching** issue by leveraging the path-based semantic information to guide the subgraph matching procedure. This is a common issue in real-world query retrieval scenarios, where the query graph and candidate data graph can be semantically equal but not topologically the same. The existing subgraph matching methods fail to handle this issue since they only consider the structural information of the graph, while our method can handle this issue by leveraging the path-based semantic information. According to the experiments in R-Table 2 (Table 5), our method achieves better performance than the subgraph matching baselines. We have added the related discussion in Section 2 and Section 4.6 of our revised paper.

---

**Meta-R3:** *Performance and Cost Tradeoff Analysis vs. Classic RAG: The paper must analyze whether MGRAG’s benefits justify its added complexity and overhead compared to standard RAG baselines.*

What is required

- Empirical comparison of MGRAG against classic/vanilla RAG approaches, including both effectiveness and cost (e.g., latency, construction overhead).
- Explicit discussion of performance–cost tradeoffs.

How this will be evaluated

- Experiments include clear baselines and comparable settings.
- Results demonstrate either meaningful gains or a well-justified tradeoff, supported by quantitative analysis rather than anecdotal claims.

**Reply to Meta-R3:** Thanks for your insightful suggestions. In our revised paper, we have revised the following of our paper according to your suggestions:

- In our revised paper, we compare our MGRAG with the classic VanillaRAG on different hop queries from the MRAMG [61] benchmark, and the result is listed in R-Table 8 (Table 9). Moreover, we also add an in detailed cost analysis of our MGRAG in R-Table 7 (Table 8), which includes the breakdown latency of the multimodal KG construction (MMKGC), coarse grained retrieval (Coarse), semantic path-based subgraph matching (Matching), and the final LLM QA (QA) component of our MGRAG. All these experiments are conducted under the same setting as our main experiment in our paper, which uses the Qwen2.5-VL-7B-Instruct LLM backbone.

- We provide quantitative analysis on the performance-cost trade-off of our MGRAG compared to VanillaRAG in Section 4.2 of our revised paper, and list their performance on 1 to 6 hop queries of MRAMG benchmark. According to the experiment result, our MGRAG greatly surpasses the VanillaRAG on queries that requires long-hop reasoning ability. On 6-hop queries, our MGRAG can achieve an 11.6% accuracy gain compared to VanillaRAG, while cost only 0.25 mins more per query in average. This demonstrates the effectiveness of our MGRAG on long-hop query retrieval scenarios, which is a common scenario in real-world applications. It is also worth noting that our MGRAG can still achieve a 1.8% accuracy gain compared to VanillaRAG on 1-hop queries, which demonstrates the effectiveness of our MGRAG on short-hop query retrieval scenarios as well. We have added the related discussion in Section 4.2 of our revised paper.

---

**Meta-R4:** *Module-Level Ablation Study (Including KV Cache): The paper must disentangle the contributions of different modules to demonstrate where improvements actually come from.*

What is required

- Ablation studies for each major novel module (e.g., KG construction, semantic matching, KV-cache mechanism).
- In particular, compare classic RAG augmented with the KV-cache module alone to assess whether gains stem primarily from this addition.

How this will be evaluated

- Ablation results are comprehensive and systematic.
- The paper clearly explains which components contribute to which improvements, supported by experimental evidence.

**Reply to Meta-R4:** Thanks for your insightful suggestions. In our revised paper, we have revised the following of our paper according to your suggestions:

- In our revised paper, we add the ablation experiments for each novel module, including the KG construction, semantic path-based subgraph matching, and KV-cache mechanism. For KG construction, and semantic path-based subgraph matching module, the results on 7 datasets are listed in R-Table 2 (Table 5). For the graph aware KV-Cache mechanism, we add a more detailed inverse ablation experiment, comparing MGRAG’s performance with the current best performing graph-based RAG models HippoRAG2 [25] and classic VanillaRAG [20] with and without our graph-based KV cache mechanism on the MMQA and Manual datasets, the results are listed in R-Table 5 and R-Table 6 (Table 11 and Table 15), respectively. In these experiments, as our graph-aware mechanism is not functional on non-graph based VanillaRAG, we add our graph matching mechanism to HippoRAG2 and VanillaRAG to test the performance improvement brought by our graph-based KV cache mechanism. All the experiments are conducted with the Qwen2.5-VL-7B-Instruct LLM backbone.
- According to the experiment results in R-Table 2, R-Table 5 and R-Table 6 (Table 5, Table 11 and Table 15), both our path-based semantic subgraph matching method, and graph-based KV cache mechanism can contribute to the performance of our MGRAG, demonstrating the effectiveness of each module. Moreover, both

of these two modules can further improve the performance of current best performing graph-based RAG model HippoRAG2, demonstrating their generalizability and effectiveness. We have added the related discussion in Section 4.5 of our revised paper.

---

**Meta-R5:** *Clarity, Structure, and Presentation Quality: The paper must be revised for clarity, coherence, and correctness.*

What is required

- Fix grammatical errors and typos throughout.
- Improve structural clarity, including a clear correspondence between stated challenges and proposed steps (e.g., Section 3.1 vs. abstract/introduction).

How this will be evaluated

- The revised paper is clearly written and professionally presented.
- The motivation, challenges, and solutions align consistently across sections.

**Reply to Meta-R5:** Thanks for your insightful suggestions. In our revised paper, we have revised the following of our paper according to your suggestions:

- We have fixed the grammatical errors and typos throughout our revised paper.
- We have modified the Section 3.1, to make the motivation, challenges, and solutions align with our revised Abstract and Introduction part.

---

**Meta-S1:** *Experimental Comparison with Algorithmic Graph Matching Methods: Where feasible, compare against representative algorithmic subgraph-matching approaches to further strengthen positioning.*

**Reply to Meta-S1:** Thanks for your insightful suggestions. In our revised paper, we further compared our subgraph matching method with the algorithmic exact solver (VF2 [13], ISMAGS [28]), approximate match algorithms (SM [35], VJ [17], RRWM [11], IPFP [36]), and semantic match algorithms (BGE [58], SLOAlign [52], FG-WAlign [53]), and the results are listed in R-Table 2 (Table 5). In the experiment, we use the Qwen2.5-VL-7B-Instruct as the backbone and fix other component of MGRAG and only replace the subgraph matching module with these baselines to ensure a fair comparison.

The results on all 7 datasets of MMQA and MRAMG benchmarks demonstrate the better performance of our path-based semantic subgraph matching method compared to other baselines. We have added the related discussion in Section 4.4 of our revised paper.

---

**Meta-S2:** *Deeper Analysis of the Greedy Solver: Provide additional discussion or analysis of approximation behavior, failure cases, or sensitivity to graph density.*

**Reply to Meta-S2:** Thanks for your insightful suggestions. In our revised paper, we have revised the following of our paper according to your suggestions:

- **Approximation behavior:** In our revised paper, we prove our semantic path-based subgraph matching problem is NP-hard and achieves  $\theta$  approximation in Theorem 4 in Section 3.3. The related discussion is also added in Section 3.3 of our revised paper.

- **Failure cases:** In our revised paper, we add an R-Table 8 (Table 9), comparing our MGRAG’s performance with the classic VanillaRAG on different hop queries from the MRAMG [61] benchmark. According to the experiment result, on 1-hop queries, our MGRAG only achieves a 1.8% accuracy gain, but costs 0.22 minutes more times per query compared to VanillaRAG [20]. This demonstrates that our MGRAG’s improvement on long-hop reasoning ability by the integration of our semantic path-based subgraph matching method can be not that worthwhile on simple queries. We have added the related discussion in Section 4.2 of our revised paper.
- **Sensitivity to graph density:** In our revised paper, we add the experiment of our path-based semantic subgraph matching method compared with the exact solver VF2 [13] on graphs of varying densities sampled from MMQA dataset [51], the result is shown in R-Figure 5 (Figure 7). In the experiment, we utilize the Qwen2.5-VL-7B-Instruct as the backbone for the LLM-based method, and apply our graph-based KV cache mechanism to both methods to ensure a fair comparison. In R-Figure 5, we can see that our path-based semantic subgraph matching method achieves better effectiveness while has comparable runtime compared with the exact solver VF2. Due to space limitation, we add the related discussion in the technical report of our revised paper at <https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf>.

---

**Meta-S3:** *Emphasize Database Significance in the Writing: Further revise the exposition to highlight why the addressed problems are important from a database and data-management perspective, especially in the context of LLM-enabled systems.*

**Reply to Meta-S3:** Thanks for your insightful suggestions. We have revised the abstract, introduction (Section 1) and related work (Section 2) part of our revised paper to further emphasize the importance of the addressed problems from a database and data-management perspective.

In LLM-enabled systems, previous subgraph matching methods ignore the semantic and topological nuances of the generated KGs and can suffer from information loss, which makes them fail to retrieve the semantically equivalent reasoning chains with different graph topologies (Namely **inflexible graph matching** issue). For example, in R-Figure 1 (Figure 1), the LLM may extract a Grandfather\_of relation in one sample, while extract two consecutive Father\_of relations in another sample, which are semantically equivalent but topologically different. This is a common data-management issue in real-world query retrieval scenarios, especially when the KGs are created or enriched automatically with LLMs, and inherently contains the noises caused by the inflexible graph matching issue.

Hence, we propose a novel path-based semantic subgraph matching mechanism, which evaluates the similarity between the query graph and the constructed KG based on the aggregated semantic embeddings of their paths, rather than directly applying the strict topological mapping based subgraph matching.

---

**R-Table 1: The experiment result on MMQA [51], and 7 sub datasets of the MRAMG [61] benchmark, all experiments were carried out with Qwen2.5-VL-7B-Instruct backbone. "-" denotes the experiment cost longer than 3 days.**

| Dataset | Metric | Similarity-based |        | Graph-based |        |        |               |
|---------|--------|------------------|--------|-------------|--------|--------|---------------|
|         |        | Vanilla.         | R1.    | Graph.      | Light. | Hippo. | MGRAG         |
| MMQA    | SIM.   | 0.8362           | 0.5478 | -           | -      | 0.8381 | <b>0.8533</b> |
|         | ACC.   | 0.7053           | 0.3567 | -           | -      | 0.6335 | <b>0.7442</b> |
|         | ROG.   | 0.5942           | 0.1041 | -           | -      | 0.6659 | <b>0.6763</b> |
|         | BLEU   | 0.6645           | 0.0817 | -           | -      | 0.5917 | <b>0.7215</b> |
| Arxiv   | SIM.   | 0.8664           | 0.7051 | 0.7943      | 0.7404 | 0.8293 | <b>0.8760</b> |
|         | ACC.   | 0.6638           | 0.3775 | 0.2350      | 0.3813 | 0.4688 | <b>0.6800</b> |
|         | ROG.   | 0.2713           | 0.1189 | 0.1553      | 0.1580 | 0.2331 | <b>0.2726</b> |
|         | BLEU   | 0.2452           | 0.1092 | 0.1690      | 0.1370 | 0.1281 | <b>0.2498</b> |
| Manual  | SIM.   | 0.8166           | 0.6046 | 0.5453      | 0.7255 | 0.6852 | <b>0.8221</b> |
|         | ACC.   | 0.5881           | 0.2071 | 0.1436      | 0.4096 | 0.2641 | <b>0.6173</b> |
|         | ROG.   | 0.3509           | 0.1078 | 0.0974      | 0.1595 | 0.2047 | <b>0.3839</b> |
|         | BLEU   | 0.2767           | 0.0740 | 0.0718      | 0.1385 | 0.1314 | <b>0.3093</b> |
| Web     | SIM.   | 0.6937           | 0.6620 | 0.5470      | 0.5513 | 0.5642 | <b>0.7009</b> |
|         | ACC.   | 0.4703           | 0.4200 | 0.1053      | 0.2000 | 0.3607 | <b>0.5123</b> |
|         | ROG.   | 0.1508           | 0.1142 | 0.1133      | 0.0905 | 0.0616 | <b>0.1664</b> |
|         | BLEU   | 0.0326           | 0.0785 | 0.1031      | 0.0206 | 0.0133 | <b>0.1456</b> |
| Wiki    | SIM.   | 0.8824           | 0.5478 | 0.6966      | 0.7517 | 0.8204 | <b>0.8838</b> |
|         | ACC.   | 0.7800           | 0.3567 | 0.2000      | 0.3660 | 0.6500 | <b>0.7845</b> |
|         | ROG.   | 0.3559           | 0.1418 | 0.1361      | 0.2008 | 0.3042 | <b>0.3579</b> |
|         | BLEU   | 0.3053           | 0.1146 | 0.1007      | 0.1863 | 0.1851 | <b>0.3174</b> |
| Wit     | SIM.   | 0.7851           | 0.6110 | 0.6211      | 0.6973 | 0.7230 | <b>0.8033</b> |
|         | ACC.   | 0.6675           | 0.6017 | 0.1500      | 0.2483 | 0.5138 | <b>0.6967</b> |
|         | ROG.   | 0.3255           | 0.1307 | 0.1319      | 0.2416 | 0.2712 | <b>0.3589</b> |
|         | BLEU   | 0.2095           | 0.0934 | 0.1142      | 0.1928 | 0.1200 | <b>0.2382</b> |
| Recipe  | SIM.   | 0.8599           | 0.6155 | 0.8166      | 0.8118 | 0.7792 | <b>0.8673</b> |
|         | ACC.   | 0.6525           | 0.2180 | 0.2771      | 0.3334 | 0.4917 | <b>0.6814</b> |
|         | ROG.   | 0.3714           | 0.1065 | 0.1745      | 0.2128 | 0.3303 | <b>0.4220</b> |
|         | BLEU   | 0.2599           | 0.0773 | 0.1527      | 0.1648 | 0.1941 | <b>0.3102</b> |

## 2 REVIEWER 2

**R2W1:** Except the path matching mechanism and the chaining of the various methods, it is not clear what the contribution is.

**Reply to R2W1:** Thanks for your insightful suggestions. Here we list our contributions as follows, which have been further clarified in Section 1 of our revised paper.

- **Framework:** Our MGRAG applies a Coarse-to-fine grained index and retrieval framework. This make our MGRAG avoid the time-consuming multimodal KG construction over the entire source document, and only construct the query-specific KG over the retrieved relevant candidates. The framework consists of 3 stages: (1). Coarse-grained retrieval stage, which retrieves the coarse-grained chunks based on their semantic similarity with the query, and then construct the query-specific KG in with the retrieved chunks with the same LLM construction method to other graph-based RAG models; (2). Fine-grained retrieval stage, which retrieves query relevant subgraph from the constructed KG based on our **path-based semantic subgraph matching** mechanism, which considers both the path-based semantic similarity and the topological alignment between the query graph and the constructed KG; (3). LLM QA stage, where we apply our

**R-Table 2: Ablation experiments of MGRAG with 2 multimodal KG construction and 5 subgraph matching baselines, all experiments were carried out with Qwen2.5-VL-7B-Instruct LLM backbone.**

| Dataset | Metric | MMKGC   |       | Subgraph Matching |        |                   |       |       |       |                |       | Ours  |       |
|---------|--------|---------|-------|-------------------|--------|-------------------|-------|-------|-------|----------------|-------|-------|-------|
|         |        | Non LLM |       | Exact Match       |        | Approximate Match |       |       |       | Semantic Match |       |       |       |
|         |        | O.IE    | STAT  | VF2               | ISMAGS | SM                | VJ    | RRWM  | IPFF  | BGE            | SLOT  |       | FGW   |
| MMQA    | SIM.   | 0.642   | 0.655 | 0.851             | 0.833  | 0.832             | 0.812 | 0.838 | 0.773 | 0.832          | 0.838 | 0.837 | 0.853 |
|         | ACC.   | 0.571   | 0.602 | 0.735             | 0.706  | 0.702             | 0.715 | 0.698 | 0.633 | 0.703          | 0.700 | 0.697 | 0.744 |
|         | ROG.   | 0.510   | 0.519 | 0.673             | 0.638  | 0.632             | 0.646 | 0.647 | 0.601 | 0.635          | 0.650 | 0.647 | 0.676 |
|         | BLEU   | 0.483   | 0.500 | 0.620             | 0.590  | 0.582             | 0.606 | 0.599 | 0.585 | 0.586          | 0.602 | 0.598 | 0.722 |
| Arxiv   | SIM.   | 0.618   | 0.640 | 0.871             | 0.868  | 0.866             | 0.872 | 0.874 | 0.814 | 0.874          | 0.872 | 0.868 | 0.876 |
|         | ACC.   | 0.526   | 0.576 | 0.650             | 0.643  | 0.670             | 0.667 | 0.675 | 0.581 | 0.668          | 0.653 | 0.651 | 0.680 |
|         | ROG.   | 0.177   | 0.184 | 0.270             | 0.266  | 0.263             | 0.269 | 0.257 | 0.212 | 0.270          | 0.269 | 0.264 | 0.273 |
|         | BLEU   | 0.138   | 0.145 | 0.239             | 0.231  | 0.244             | 0.245 | 0.243 | 0.203 | 0.244          | 0.233 | 0.230 | 0.250 |
| Manual  | SIM.   | 0.520   | 0.509 | 0.817             | 0.817  | 0.813             | 0.808 | 0.811 | 0.800 | 0.815          | 0.813 | 0.820 | 0.822 |
|         | ACC.   | 0.459   | 0.443 | 0.612             | 0.612  | 0.614             | 0.608 | 0.614 | 0.594 | 0.615          | 0.611 | 0.600 | 0.617 |
|         | ROG.   | 0.245   | 0.234 | 0.377             | 0.382  | 0.378             | 0.372 | 0.377 | 0.369 | 0.375          | 0.375 | 0.374 | 0.384 |
|         | BLEU   | 0.209   | 0.182 | 0.306             | 0.307  | 0.303             | 0.298 | 0.303 | 0.289 | 0.303          | 0.301 | 0.292 | 0.309 |
| Web     | SIM.   | 0.466   | 0.485 | 0.677             | 0.675  | 0.676             | 0.679 | 0.675 | 0.666 | 0.687          | 0.676 | 0.676 | 0.701 |
|         | ACC.   | 0.290   | 0.298 | 0.507             | 0.509  | 0.507             | 0.508 | 0.503 | 0.493 | 0.510          | 0.498 | 0.480 | 0.512 |
|         | ROG.   | 0.052   | 0.101 | 0.156             | 0.155  | 0.156             | 0.160 | 0.157 | 0.149 | 0.156          | 0.154 | 0.147 | 0.166 |
|         | BLEU   | 0.020   | 0.095 | 0.088             | 0.079  | 0.056             | 0.056 | 0.066 | 0.055 | 0.095          | 0.116 | 0.096 | 0.146 |
| Wiki    | SIM.   | 0.504   | 0.525 | 0.877             | 0.879  | 0.878             | 0.873 | 0.878 | 0.869 | 0.878          | 0.879 | 0.878 | 0.884 |
|         | ACC.   | 0.498   | 0.510 | 0.752             | 0.754  | 0.750             | 0.751 | 0.743 | 0.735 | 0.746          | 0.749 | 0.747 | 0.785 |
|         | ROG.   | 0.230   | 0.253 | 0.352             | 0.355  | 0.353             | 0.349 | 0.355 | 0.348 | 0.355          | 0.356 | 0.352 | 0.358 |
|         | BLEU   | 0.200   | 0.218 | 0.300             | 0.303  | 0.298             | 0.297 | 0.301 | 0.296 | 0.301          | 0.301 | 0.300 | 0.317 |
| Wit     | SIM.   | 0.492   | 0.489 | 0.799             | 0.797  | 0.784             | 0.786 | 0.795 | 0.792 | 0.789          | 0.793 | 0.794 | 0.803 |
|         | ACC.   | 0.465   | 0.453 | 0.693             | 0.694  | 0.686             | 0.691 | 0.679 | 0.678 | 0.689          | 0.693 | 0.691 | 0.697 |
|         | ROG.   | 0.233   | 0.205 | 0.354             | 0.355  | 0.353             | 0.356 | 0.347 | 0.346 | 0.356          | 0.351 | 0.349 | 0.359 |
|         | BLEU   | 0.210   | 0.198 | 0.234             | 0.231  | 0.229             | 0.227 | 0.230 | 0.231 | 0.230          | 0.227 | 0.223 | 0.238 |
| Recipe  | SIM.   | 0.551   | 0.545 | 0.859             | 0.861  | 0.862             | 0.857 | 0.852 | 0.847 | 0.864          | 0.861 | 0.860 | 0.867 |
|         | ACC.   | 0.425   | 0.401 | 0.679             | 0.668  | 0.678             | 0.672 | 0.670 | 0.661 | 0.674          | 0.675 | 0.677 | 0.681 |
|         | ROG.   | 0.211   | 0.198 | 0.413             | 0.407  | 0.411             | 0.413 | 0.406 | 0.395 | 0.415          | 0.412 | 0.409 | 0.422 |
|         | BLEU   | 0.202   | 0.172 | 0.304             | 0.302  | 0.305             | 0.295 | 0.298 | 0.221 | 0.308          | 0.306 | 0.305 | 0.310 |

**R-Table 3: The statistic of MMQA [51], and 7 sub dataset of the MRAMG [61] benchmark. T and I in Query Modality denotes the dataset contains textual and image queries, respectively.**

| Statistic      | MMQA    | Wiki | Wit  | Recipe | Arxiv | Manual | Web   |
|----------------|---------|------|------|--------|-------|--------|-------|
| Query Modality | T / I   | T    | T    | T      | T     | T      | T     |
| Queries        | 951     | 500  | 600  | 2,360  | 200   | 390    | 750   |
| Texts          | 218,285 | 538  | 639  | 1,528  | 101   | 40     | 1,500 |
| Images         | 57,058  | 538  | 639  | 8,569  | 337   | 2,607  | 1,500 |
| Corpus Size    | 22.5M   | 680k | 478k | 1.1M   | 136k  | 420k   | 215k  |
| Query Hops     | 1-2     | 2-5  | 2-4  | 1-6    | 3-4   | 2-6    | 2-3   |
| Avg Query Hops | 1.5     | 3.0  | 2.8  | 3.3    | 3.1   | 3.7    | 2.4   |

**R-Table 4: Efficiency and effectiveness comparasion between MGRAG and other KV cache mechanisms on MMQA dataset. Where V. and H. denote these systems cope with similarity-based VanillaRAG and graph-based HippoRAG2 framework.**

| System          | SIM.          | ACC.          | ROG.          | BLEU          | TTFT         | Time Cost    |
|-----------------|---------------|---------------|---------------|---------------|--------------|--------------|
| vLLM (V.)       | 0.8315        | 0.6593        | 0.6277        | 0.5867        | 3.2 s        | 4.5 h        |
| CacheBlend (V.) | 0.8315        | 0.6601        | 0.6417        | 0.5964        | 3.1 s        | 4.5 h        |
| vLLM (H.)       | 0.8381        | 0.6335        | 0.6659        | 0.5917        | 4.6 s        | 8.8 h        |
| CacheBlend (H.) | 0.8415        | 0.6267        | 0.6636        | 0.5917        | 4.4 s        | 8.3 h        |
| <b>MGRAG</b>    | <b>0.8533</b> | <b>0.7442</b> | <b>0.6763</b> | <b>0.7215</b> | <b>3.2 s</b> | <b>4.3 h</b> |

**graph-aware KV caching** mechanism, which scores the KV cache based on the retrieved subgraph and the attention weights, further leverage the graph topological information to guide the KV cache reuse procedure.

- **Path-based semantic subgraph matching mechanism:** According to R-Figure 1 (Figure 1), current subgraph matching methods [11, 13, 17, 28, 35, 36, 52, 53] suffer from the **inflexible graph matching issue** as they ignore the potential match when the retrieved subgraph is topologically different but semantically similar to the query graph. This is common in the LLM-generated KGs due to the noise and information loss in the KG construction process [57, 70]. In our MGRAG, we propose a novel path-based semantic subgraph matching mechanism, which addresses the inflexible graph matching issue by evaluating the similarity between the query graph and the constructed KG based on the aggregated semantic embeddings of their paths. Denote the similarity threshold as  $\theta$ , our algorithm achieves a  $\theta$ -approximation of the optimal subgraph matching.
- **Graph-aware KV caching mechanism:** In graph-based RAG scenarios, the current KV cache mechanism [5, 34, 60] ignores the topological importance of certain token’s respective entity/relation within the graph. In our MGRAG, we propose a novel graph-based KV cache mechanism, which scores the KV cache based on both of the retrieved subgraph’s topological features and the LLM attention weights, further leverage the graph topological information. According to R-Table 9 (Table 7), our graph-based KV cache mechanism can further improve the effectiveness of current best-performing graph-based RAG models HippoRAG2 [25], demonstrating its generalizability and effectiveness.
- **Experiments:** In our revised paper, we add experiments comparing our path-based semantic subgraph matching mechanism with other subgraph matching baselines, including exact solver (VF2 [13], ISMAGS [28]), approximate match algorithms (SM [35], VJ [17], RRWM [11], IPFP [36]), and semantic match algorithms (BGE [58], SLOAlign [52], FGWAlign [53]). We list these results in the R-Table 2 (Table 5) in our revised paper. Furthermore, we test our graph-based KV cache mechanism against other baselines, including vLLM [34] and CacheBlend [60], on the MMQA dataset, and the results are listed in R-Table 9 (Table 7). Moreover, we add inverse ablation experiments by adding our path-based semantic subgraph matching mechanism and graph-based KV cache mechanism to the best-performing RAG models HippoRAG2 [25] and VanillaRAG [20], and the results are listed in R-Table 5 (Table 11), and R-Table 6 (Table 15) for MMQA and Manual dataset. These results demonstrate the effectiveness and efficiency of our MGRAG, and also show that other RAG baselines can also benefit from our proposed mechanisms.

**R2D1:** Out of the three parts of the approach, the most novel seems to be the fine-grained retrieval based on evaluating paths in the graph. However, (isomorphism-based) path matching in graphs is a well-studied problem, and the paper does not make clear what exactly is new in the "LLM world" compared to classic graph algorithms.

**Reply to R2D1:** Thanks for your insightful suggestions. In the "LLM world", previous subgraph matching methods ignore the semantic and topological nuances of the generated KGs and can suffer from

**R-Table 5: The inverse ablation of MGRAG’s path-based semantic subgraph matching (PSGM) and graph-based KV (GKV) cache mechanism, with two best performing RAG system HippoRAG2 [25] and VanillaRAG [20] on MMQA.**

| Model             | SIM.          | ACC.          | ROG.          | BLEU          | Time         |
|-------------------|---------------|---------------|---------------|---------------|--------------|
| Hippo.            | 0.8381        | 0.6335        | 0.6659        | 0.5917        | 7.5 h        |
| Hippo.+GKV        | 0.8392        | 0.6855        | 0.6651        | 0.7013        | 7.6 h        |
| Hippo.+PSGM       | 0.8416        | <u>0.7372</u> | <u>0.6720</u> | <u>0.7128</u> | <u>4.1 h</u> |
| Hippo.+PSGM+GKV   | <b>0.8533</b> | <b>0.7442</b> | <b>0.6763</b> | <b>0.7215</b> | 4.3 h        |
| Vanilla.          | 0.8362        | 0.7053        | 0.5942        | 0.6645        | <b>2.6 h</b> |
| Vanilla.+PSGM     | 0.8460        | 0.7175        | 0.6466        | 0.6973        | 4.7 h        |
| Vanilla.+PSGM+GKV | 0.8485        | 0.7232        | 0.6563        | 0.7001        | 4.8 h        |
| <b>MGRAG</b>      | <b>0.8533</b> | <b>0.7442</b> | <b>0.6763</b> | <b>0.7215</b> | 4.3 h        |

information loss, which makes them fail to retrieve the semantically equivalent reasoning chains with different graph topologies (Namely **inflexible graph matching issue**). For example, as shown in R-Figure 1 (Figure 1), the LLM may extract a Grandfather\_of relation in the query graph, while extract two consecutive Father\_of relations in the data graph, which are semantically equivalent but topologically different. Hence, we propose a novel path-based semantic subgraph matching mechanism, which evaluates the similarity between the query graph and the constructed KG based on the aggregated semantic embeddings of their paths, rather than directly applying the strict topological mapping based subgraph matching.

In the revised paper, we add the demonstration of current subgraph matching baselines [11, 13, 17, 28, 35, 36, 52, 53] in our related work part (Section 2). Moreover, we also add R-Table 2 (Table 5), comparing our path-based semantic subgraph matching mechanism’s performance with other subgraph matching baselines on MMQA [51] dataset and 6 datasets from MRAMG [61] benchmark with Qwen2.5-VL-7B Instruct LLM, including exact solver (VF2 [13], ISMAGS [28]), approximate match algorithms (SM [35], VJ [17], RRWM [11], IPFP [36]), and semantic match algorithms (BGE [58], SLOAlign [52], FGWAlign [53]). The experiment results demonstrate the effectiveness of our path-based semantic subgraph matching mechanism.

**R2W2, R2D3:** The comparison with the baselines may be slightly unfair. There’s nothing that I can see that make it impossible to add the proposed retrieval modules and the KV caches to the vanilla RAG (or conversely, add RAGs to KVs, etc.) In other words, a fairer comparison would have been an "inverse ablation" study where the proposed components would be added to the state-of-the-art. Seeing the current results, these new experiments may show some surprises.

**Reply to R2W2, R2D3:** Thanks for your insightful suggestions. Our MGRAG consists of three stages, namely multimodal KG construction, path-based semantic subgraph matching, and graph-based KV cache mechanism. In our revised paper, we add inverse ablations of MGRAG’s three stages on the MMQA (the easiest) dataset and Manual (the hardest) dataset from the MRAMG benchmark in R-Table 5 and R-Table 6 (Table 11 and Table 15), where we integrate our path-based semantic subgraph matching (Namely PSGM) and

**R-Table 6: The inverse ablation of MGRAG’s path-based semantic subgraph matching (PSGM) and graph-based KV (GKV) cache mechanism, with two best performing RAG model HippoRAG2 [25] and VanillaRAG [20] on Manual.**

| Model             | SIM.          | ACC.          | ROG.          | BLEU          | Time         |
|-------------------|---------------|---------------|---------------|---------------|--------------|
| Hippo.            | 0.6852        | 0.2641        | 0.2047        | 0.1314        | 2.4 h        |
| Hippo.+GKV        | 0.6808        | 0.2710        | 0.1998        | 0.1425        | 2.4 h        |
| Hippo.+PSGM       | 0.7958        | 0.5691        | 0.3741        | 0.2950        | 1.9 h        |
| Hippo.+PSGM+GKV   | <b>0.8221</b> | <b>0.6173</b> | <b>0.3839</b> | <b>0.3093</b> | 1.9 h        |
| Vanilla.          | 0.8166        | 0.5881        | 0.3509        | 0.2767        | <b>0.8 h</b> |
| Vanilla.+PSGM     | 0.8195        | 0.6003        | 0.3781        | 0.2975        | 2.0 h        |
| Vanilla.+PSGM+GKV | 0.8203        | 0.6095        | 0.3758        | 0.3010        | 2.0 h        |
| <b>MGRAG</b>      | <b>0.8221</b> | <b>0.6173</b> | <b>0.3839</b> | <b>0.3093</b> | <b>1.8 h</b> |

**R-Table 7: MGRAG’s breakdown time cost for each module. Where KGC column denotes multimodal KG construction time and token cost, Match column denotes subgraph matching time cost, QA column denotes LLM QA time and token cost, and E2E denotes end-to-end time and token cost.**

| Dataset | # Query | KGC           | Match | QA            | E2E            |
|---------|---------|---------------|-------|---------------|----------------|
| MMQA    | 951     | 193.0 / 3,085 | 0.14  | 8.9 / 1,431   | 248.1 / 4,516  |
| Arxiv   | 200     | 61.9 / 3,946  | 0.23  | 16.2 / 2,944  | 83.9 / 6,890   |
| Manual  | 390     | 76.1 / 4,285  | 0.04  | 23.9 / 2,556  | 109.7 / 6,841  |
| Web     | 750     | 143.8 / 4,623 | 0.27  | 42.9 / 3,101  | 196.5 / 7,723  |
| Wiki    | 500     | 164.2 / 6,629 | 0.30  | 52.1 / 6,267  | 241.5 / 12,896 |
| Wit     | 600     | 140.6 / 7,660 | 0.21  | 42.6 / 6,539  | 196.5 / 14,199 |
| Recipe  | 2,360   | 525.7 / 4,351 | 1.74  | 191.3 / 3,173 | 808.7 / 7,525  |

graph-based KV cache (Namely GKV) mechanism with the previously best-performed graph-based RAG model HippoRAG2 [25], and non graph-based model VanillaRAG [20]. The experiments were carried out using the Qwen2.5-VL-7B Instruct LLM backbone, aligning with the experiments in our paper.

According to R-Table 5 and R-Table 6 (Table 11 and Table 15), each of our proposed path-based semantic subgraph matching (PSGM) and graph-based KV cache mechanism (GKV) can further improve the performance of these two tested baselines, and achieve the best performance when combined, demonstrating the effectiveness and generalizability of our proposed mechanisms. Moreover, it is also worth noting that the performance of HippoRAG2+PSGM+GKV is the same as our MGRAG, since our MGRAG applied the same KG construction method as HippoRAG2, HippoRAG+PSGM+GKV is just equals to our MGRAG.

**R2W3:** The cost (latency) should be studied more in depth.

**Reply to R2W3:** Thank you for your insightful suggestions. In our revised paper, we provide a breakdown time cost of the multimodal KG construction (MMKGC), coarse grained retrieval (Coarse), semantic path-based subgraph matching (Matching), and the final LLM QA (QA) component of our MGRAG in R-Table 7 (Table 8). In the experiment, we use the Qwen2.5-VL-7B Instruct LLM backbone, aligning with the experiments in our paper.

**R-Table 8: Performance comparison between VanillaRAG and MGRAG on different hop queries of MRAMG benchmark with Qwen2.5-VL-7B-Instruct LLM backbone. The times listed are per query time in minutes.**

| # Hop | # Q   | VanillaRAG  |       |       |       | MGRAG |              |              |              |
|-------|-------|-------------|-------|-------|-------|-------|--------------|--------------|--------------|
|       |       | Time        | ACC.  | ROG.  | BLEU  | Time  | ACC.         | ROG.         | BLEU         |
| 1-hop | 201   | <b>0.13</b> | 0.613 | 0.379 | 0.240 | 0.35  | <b>0.624</b> | <b>0.425</b> | <b>0.286</b> |
| 2-hop | 1,237 | <b>0.10</b> | 0.588 | 0.304 | 0.193 | 0.37  | <b>0.601</b> | <b>0.335</b> | <b>0.230</b> |
| 3-hop | 2,357 | <b>0.16</b> | 0.633 | 0.329 | 0.238 | 0.41  | <b>0.659</b> | <b>0.355</b> | <b>0.260</b> |
| 4-hop | 499   | <b>0.19</b> | 0.697 | 0.338 | 0.259 | 0.44  | <b>0.712</b> | <b>0.373</b> | <b>0.293</b> |
| 5-hop | 286   | <b>0.18</b> | 0.678 | 0.319 | 0.205 | 0.45  | <b>0.733</b> | <b>0.392</b> | <b>0.282</b> |
| 6-hop | 220   | <b>0.18</b> | 0.661 | 0.305 | 0.176 | 0.43  | <b>0.738</b> | <b>0.377</b> | <b>0.249</b> |

As shown in R-Table 7 (Table 8), the key bottleneck of MGRAG is its multimodal KG construction step, which is also a key bottleneck for all current graph-based RAG models [12] and a cost for their long hop reasoning ability. On the other hand, our novel path-based semantic subgraph matching only cost a small margin of the total time cost, i.e., 0.06%-0.5%, demonstrating the efficiency of our proposed greedy algorithm. We have added the related analysis to Section 4.3 in our revised paper.

**R2D2:** Per-token cost seems to be good enough. However, the entire workload can be costlier than the classic RAG approaches, with not that much difference in accuracy (vanilla based RAGs are usually second). This trade-off needs to be better explained.

**Reply to R2D2:** Thank you for your insightful suggestions. We also add a more detailed discussion on the trade-off between the cost and performance improvement of our MGRAG compared to the VanillaRAG in Section 4.2. In R-Table 8 (Table 9), we compare MGRAG and VanillaRAG on queries requiring different hops of knowledge from the MRAMG [61] benchmark. In the experiment, we use the Qwen2.5-VL-7B-Instruct LLM backbone, aligning with our paper’s main experiment.

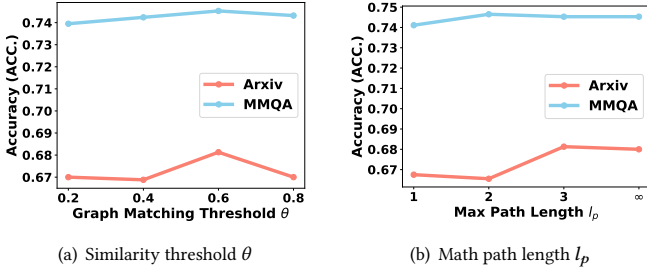
As shown in R-Table 8 (Table 9), MGRAG’s advantage over VanillaRAG grows with query complexity: the accuracy gain rises from 1.8% on 1-hop queries to 11.6% on 6-hop queries. Although MGRAG increases query time by about 0.25 minutes per long query, this is an acceptable trade-off for its substantial gains in long-hop reasoning. We have added the related discussion in Section 4.2 of our revised paper.

**R2D4:** The paper is littered with grammar and spelling errors, especially in the experimental section. Some examples: "where the MMQA dataset have a large number or source document", "we can have following observations", "For our MGRAG, and the length of text chunks to 256, and token overlap between chunks [...]", "although lack the ability of fine-grained retrieval", "mechaism", etc.

**Reply to R2D4:** Thank you for your comment. In our revised paper, we fixed all the grammar and spelling errors.

**R-Table 9: Efficiency and effectiveness comparasion between MGRAG and other KV cache mechanisms on MMQA dataset. Where V. and H. denote these systems cope with similarity-based VanillaRAG and graph-based HippoRAG2 framework.**

| System          | SIM.   | ACC.   | ROG.   | BLEU   | TTFT  | Time Cost |
|-----------------|--------|--------|--------|--------|-------|-----------|
| vLLM (V.)       | 0.8315 | 0.6593 | 0.6277 | 0.5867 | 3.2 s | 4.5 h     |
| CacheBlend (V.) | 0.8315 | 0.6601 | 0.6417 | 0.5964 | 3.1 s | 4.5 h     |
| vLLM (H.)       | 0.8381 | 0.6335 | 0.6659 | 0.5917 | 4.6 s | 8.8 h     |
| CacheBlend (H.) | 0.8415 | 0.6267 | 0.6636 | 0.5917 | 4.4 s | 8.3 h     |
| MGRAG           | 0.8533 | 0.7442 | 0.6763 | 0.7215 | 3.2 s | 4.3 h     |



**R-Figure 2: Parameter sensitivity experiment of MGRAG, where we fix  $l_p$  to 3, and  $\theta$  to 0.6, for testing the sensitivity of  $\theta$  and  $l_p$ , respectively.**

### 3 REVIEWER 4

**R4W1:** While the NP-hard proof and greedy solution for path-based matching are strong, there’s no analysis of approximation guarantees or failure cases (e.g., on dense vs. sparse graphs). This could be addressed by adding theoretical bounds or sensitivity tests to path length  $l_p$  to strengthen claims of “efficient” solving.

**Reply to R4W1:** Thanks for your insightful suggestions. In the revised version, we add theoretical bounds for the path-based subgraph matching algorithm (Algorithm 1) and sensitivity tests for path length  $l_p$ , detailed as follows:

- (1) In our revised Theorem 4, we analyze the approximation ratio of our path-based subgraph matching algorithm (Algorithm 1). Specifically, we prove that the algorithm’s approximation ratio is bounded by the predefined similarity threshold  $\theta \in (0, 1)$ . The parameter  $\theta$  serves as a hyperparameter: increasing  $\theta$  enforces stricter semantic matching for node and edge match. As shown in our sensitivity analysis in R-Figure 2 (a) (Figure 5 (a)), MGRAG achieves optimal performance with  $\theta = 0.6$  on both MMQA and Arxiv datasets. This suggests that a moderate semantic matching threshold balances approximation quality with the success rate of matching.
- (2) In our revised paper, we also provide a sensitivity test to the path length  $l_p$  in Section 4.6 in R-Figure 2 (b) (Figure 5 (b)), the result shows our model tends to achieve its best performance when the path length  $l_p$  is set to 2 or 3 on MMQA and Arxiv datasets. It indicates that in most of the inflexible graph matching scenarios, a particular relation can be semantically decomposed into 2 or 3 consecutive relations.

**R-Table 10: Ablation experiment of MGRAG on MMQA.**

| Model                | SIM.   | ACC.   | ROG.   | BLEU   | Time Cost |
|----------------------|--------|--------|--------|--------|-----------|
| MGRAG w.o. GM        | 0.8362 | 0.7260 | 0.6657 | 0.5774 | 4.4 h     |
| MGRAG w. MM Encoder  | 0.8513 | 0.7376 | 0.6744 | 0.7008 | 4.5 h     |
| Vanilla MGRAG w. k=5 | 0.8260 | 0.6887 | 0.6266 | 0.5773 | 4.7 h     |
| MGRAG                | 0.8533 | 0.7442 | 0.6763 | 0.7215 | 4.3 h     |

**R-Table 11: The scoring function of KV caching methods, where Prefix denotes the tokens in the input prefix,  $KV^{full}[t]$  and  $KV[t]$  denotes the fully computed, and previously cached KV attention matrices of token  $t$ , respectively.**

| Method          | Feature                   | Scoring Function $\phi(t)$                                     |
|-----------------|---------------------------|----------------------------------------------------------------|
| vLLM [34]       | Position Dependent        | $1[t \in \text{Prefix}]$                                       |
| CacheBlend [60] | Position Independent      | $\ KV[t] - KV^{full}[t]\ _2$                                   |
| MGRAG           | Graph Structure Dependent | $\sum_{j=1}^{ q } a_{tj} + \sum_{k=1}^{ S_r } a_{tk} + GF_t^q$ |

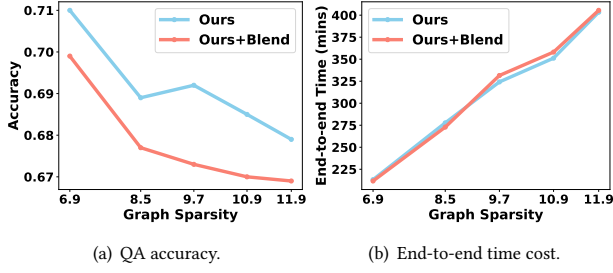
**R4W2:** The paper uses image caption as a proxy for multimodal input in the coarse-grained ranking step and simplifies images to text embeddings, potentially losing visual semantics that a true end-to-end visual encoder might capture, leading to suboptimal chunk ranking if captions are inaccurate or incomplete.

**Reply to R4W2:** Thank you for your valuable comments. Current multimodal encoders have weaker representation ability than well-developed text encoders, largely due to their limited shared embedding space (e.g., only 768 dimensions for the widely used CLIP ViT-L/14 [46] to represent both visual and textual information). In contrast, text embedding models, benefiting from advances in large language models, offer stronger representations with larger embedding dimensions (e.g., 1024 for Bge-large-en-v1.5 solely for text). This superior representation ability outweighs the information loss from the modality gap, making image captions a more effective retrieval proxy than visual encoders [22].

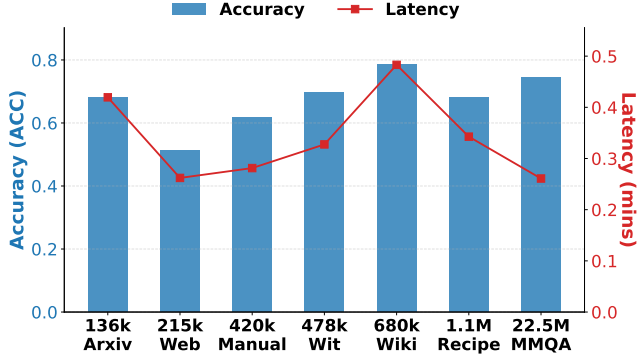
In Section 4.6 of our revised paper, we add an ablation experiment in R-Table 10 (Table 10) comparing MGRAG’s performance with the visual encoder (namely MGRAG w. MM Encoder) and the ‘image caption as a proxy’ setting. In this setting, we use the widely adopted CLIP ViT-L/14 [46] multimodal encoder to directly encode images and texts into a unified representation space for coarse-grained retrieval, without any image-to-text conversion. The results in R-Table 10 (Table 10) show that MGRAG performs better when using image captions as a proxy, aligning with previous findings [22]. It is worth noting that all tested baselines in our paper also adopt the “image caption as a proxy” setting, ensuring a fair comparison. We have added the related discussion in Section 4.6.

**R4W3:** The event-aware mechanism tunes empirically but lacks justification for why graph-topological scores  $GF_q^t$  outperform alternatives (e.g., attention-based). May expand with comparisons to token-level methods in CacheBlend, as fixed assumptions may not generalize to varying graph densities.

**Reply to R4W3:** Thanks for your insightful suggestions. According to the R-Table 11 (Table 2), our graph-based KV cache mechanism



R-Figure 3: Accuracy and end-to-end time cost comparison of our MGRAG cope with our graph-based KV cache mechanism and CacheBlend (Blend), on graphs with varying sparsities sampled from MMQA dataset.



R-Figure 4: Accuracy and per-query latency of MGRAG on datasets with various corpus size.

also considers the attention scores in scoring function, but we further integrate the graph-topological scores  $GF_q^t$  to compute the final importance score of each token, which can score the KV Caches consider their importance in the graph structure, better suited for graph-based RAG data management scenarios.

In our revised paper, we compare our proposed graph-based KV cache mechanism with CacheBlend [60]. Specifically, we replace our KV cache algorithm with the KV cache algorithm from CacheBlend [60], resulting in an “Ours+Blend” variant. We then sample five datasets with varying graph sparsity from the MMQA dataset, using Qwen2.5-VL-7B-Instruct as the backbone. Here, denote the number of graph nodes as  $|V|$ , and the number of edges as  $|\mathcal{E}|$ , we define the graph sparsity as  $\frac{|V|^2}{|\mathcal{E}|}$ , the higher this value is, the larger and sparser the graph is. The result is shown in R-Figure 3 (Figure 8), where we can see that our graph-based KV cache mechanism consistently outperforms CacheBlend across different graph sparsity, while having comparable time cost, demonstrating its better generalizability to varying graph sparsity. We have added the related discussion in Section 4.6 of our revised paper.

**R4D2:** Experiments lack real-time latency under varying corpus sizes, which is key for systems papers.

**R-Table 12:** Comparison of MGRAG pipeline cope with our graph-based KV cache mechanism and attention-based CacheBlend (Blend) [60] on 6 datasets from MRAMG [61] benchmark, the time is the end-to-end time in minutes.

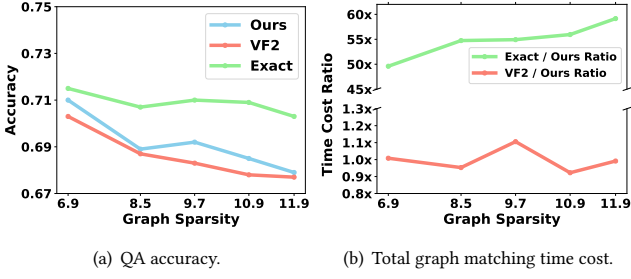
| Method       | Metric | Wiki          | Wit           | Recipe        | Arxiv         | Manual        | Web           |
|--------------|--------|---------------|---------------|---------------|---------------|---------------|---------------|
| Ours + Blend | SIM.   | 0.8792        | 0.8011        | 0.8585        | 0.8651        | 0.8208        | 0.6772        |
|              | ACC.   | 0.7635        | 0.6908        | 0.6789        | 0.6263        | 0.6090        | 0.5035        |
|              | ROG.   | 0.3563        | 0.3559        | 0.4148        | 0.2616        | 0.3772        | 0.1651        |
|              | BLEU.  | 0.3018        | 0.2366        | 0.3019        | 0.2259        | 0.2992        | 0.1307        |
|              | Time   | 246.0         | <b>180.9</b>  | 816.2         | 84.1          | <b>109.3</b>  | 204.0         |
| Ours         | SIM.   | <b>0.8838</b> | <b>0.8033</b> | <b>0.8673</b> | <b>0.8760</b> | <b>0.8221</b> | <b>0.7009</b> |
|              | ACC.   | <b>0.7845</b> | <b>0.6967</b> | <b>0.6814</b> | <b>0.6800</b> | <b>0.6173</b> | <b>0.5123</b> |
|              | ROG.   | <b>0.3579</b> | <b>0.3589</b> | <b>0.4220</b> | <b>0.2726</b> | <b>0.3839</b> | <b>0.1664</b> |
|              | BLEU.  | <b>0.3174</b> | <b>0.2382</b> | <b>0.3102</b> | <b>0.2498</b> | <b>0.3093</b> | <b>0.1456</b> |
|              | Time   | <b>241.5</b>  | 196.5         | <b>808.7</b>  | <b>83.9</b>   | 109.7         | <b>196.5</b>  |

**Reply to R4D2:** Thanks for your insightful suggestions. In our revised Section 4.3, we report the average per-query latency and accuracy of MGRAG on datasets with varying corpus sizes. The corpus size refers to the total number of tokens from both text and images in each dataset. As shown in R-Figure 4 (Figure 6) and R-Table 3 (Table 3), the per-query latency exhibits some fluctuation across different corpus sizes, but remains within a relatively narrow range (approximately 0.25 to 0.45 minutes). Our MGRAG system maintains low latency as the corpus size increases because it first retrieves a small number of relevant chunks. This retrieval step is efficient and usually takes less than one minute for all queries, as shown in R-Table 7. The MMQA dataset, which has the largest corpus, actually achieves the lowest per-query latency. This occurs because MMQA queries are simpler, requiring on average only 1.5-hop knowledge according to R-Table 3. Therefore, fewer chunks are retrieved and a smaller query-specific knowledge graph is constructed, which reduces the time required for subgraph matching and LLM-based question answering.

**R4A1:** Expand Section 2.3 (KV caching) with a figure/table comparing  $GF_q^t$  scores to cross-attention baselines on a sample dataset.

**Reply to R4A1:** Thank you for your suggestions. In our revised paper, we include R-Table 11 (Table 2 in Section 2.3) to compare the scoring function of our graph-based KV cache with other mechanisms, including vLLM [34] and CacheBlend [60]. In revised Section 4.6, we also compare our method with attention-based CacheBlend [60] in the MGRAG pipeline on six MRANG benchmark datasets, using the Qwen2.5-VL-7B-Instruct backbone. Specifically, we replace our KV cache algorithm with that of CacheBlend, resulting in an “Ours+Blend” variant. For fairness, only the KV cache mechanism varies. Results in R-Table 12 (Table 14) show that our approach consistently outperforms CacheBlend KV cache mechanism with similar end-to-end time cost, demonstrating its effectiveness and efficiency.

**R4A2:** Add a subsection in Section 4 with ablation on greedy matching. Include runtime/accuracy vs. exact solvers on synthetic graphs of varying densities.



**R-Figure 5: Accuracy and time cost comparison between our path-based semantic subgraph matching method and the exact solver VF2 [13] on graphs with varying densities sampled from MMQA dataset, where the x-axis denotes the graph density, y-axis denotes the QA accuracy and time cost.**

**Reply to R4A2:** Thanks for your insightful suggestions. In our revised paper, we add an experiment in Section 4.6 comparing our path-based semantic subgraph matching method against two baselines on graphs of varying sparsity sampled from the MMQA dataset: (1) the original VF2 [13] algorithm, which performs strict topological matching without path-level semantic aggregation, and (2) a VF2-based exact solver, which enumerates all candidate paths via depth-first search and selects the one with the highest average semantic score, representing the optimal solution to our problem. We define graph sparsity as  $\frac{|V|^2}{|E|}$ , where higher values indicate larger and sparser graphs. All experiments use Qwen2.5-VL-7B-Instruct as the backbone with  $\theta = 0.6$ , aligning with our main experiment setting.

As shown in R-Figure 5 (Figure 7), the exact solver achieves the highest accuracy but is 50–60× slower than our method, while our method consistently outperforms VF2 in accuracy with comparable runtime (0.9–1.1×). This confirms that our greedy algorithm offers a strong efficiency-effectiveness trade-off: it addresses inflexible graph matching issue that VF2 cannot, with negligible overhead, and scales favorably as graphs become sparser. We have added the related discussion in Section 4.6 of our revised paper.

**R4D1:** Assumes LLM reliability for extraction without quantification.

**Reply to R4D1:** Thank you for your valuable suggestion. In our revised paper, we add an experiment in R-Table 2 (Table 5) comparing MGRAG’s LLM-based extraction and KG construction performance with other KG construction methods, including Stanford OpenIE [6] (Namely O.IE) and a term frequency-based method applied by AGRAG [57] (Namely STAT). In the experiment, we use the Qwen2.5-VL-7B-Instruct as the backbone for LLM-based method, and apply our path-based semantic subgraph matching method and graph-based KV cache mechanism to all tested KG construction baselines to ensure a fair comparison. The results demonstrate the better effectiveness of the LLM-based method we applied compared to other non-LLM based methods, where the LLM-based method achieves 21.2% higher QA accuracy across different datasets.

Furthermore, to quantitatively evaluate the result, we apply human evaluation to the performance of the LLM-based method KG construction method we applied, with the most advanced non-LLM statistic-based KG construction methods [57] (namely STAT), which

**R-Table 13: System cost on MMQA dataset, where "-" denotes the experiment cost longer than 3 days, and AVG Token Cost denotes the average token cost per query.**

| Category         | Model       | Time Cost | AVG Token Cost |
|------------------|-------------|-----------|----------------|
| Similarity-based | VanillaRAG  | 2.6 h     | 1132.1         |
|                  | R1-Searcher | 2.9 h     | 1835.4         |
| Graph-based      | GraphRAG    | -         | -              |
|                  | LightRAG    | -         | -              |
|                  | HippoRAG2   | 7.5 h     | 292,436.2      |
|                  | MGRAG       | 4.3 h     | 4613.4         |

applies a term frequency-based entity extraction method to extract KG entities for KG construction. In the experiment, we ask human annotators to evaluate the quality of the extracted KG entities from these two methods on the same 100 random query-specific KG samples from MMQA dataset. We apply two metrics for evaluation: (1) Precision, which measures the proportion of correctly extracted entities among all extracted entities; (2) Recall, which measures the proportion of correctly extracted entities among all entities in the ground truth. According to our evaluation, the LLM-based method achieves 89.92% precision and 88.17% recall, while the statistic-based method achieves 67.20% precision and 94.82% recall. The result shows that the LLM-based method achieves 33.8% higher precision in average than the statistic-based method for KG entity extraction, demonstrating the better quality of the constructed KG by the LLM-based method. Although the statistic-based method has a 7.5% higher recall than the LLM-based method, it is due to the fact that the statistic-based method tends to extract more useless entities, which can lead to a higher recall but also more noises in the extracted KG.

**R4D3:** Typos and Ambiguous annotations.

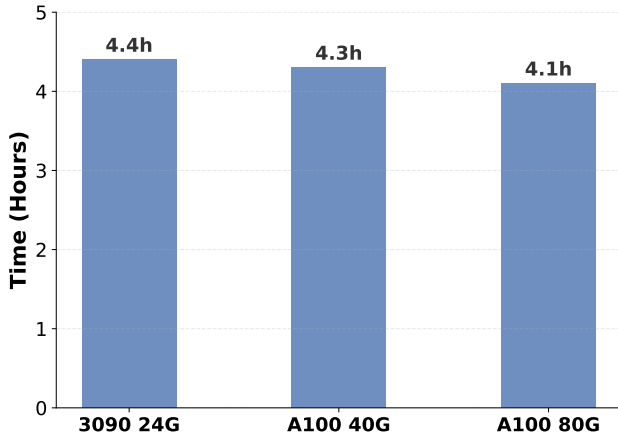
**Reply to R4D3:** Thank you for your valuable suggestion. In our revised paper, we fixed all the typos and ambiguous annotations.

## 4 REVIEWER 5

**R5W1, R5D1:** Limited discussion of LLM-related costs: The query-specific KG relies on repeated entity and relation extraction using an LLM. The paper does not deeply analyze the cost, latency, and resources needed of these repeated LLM calls, which could be substantial in large-scale deployments.

**Reply to R5W1, R5D1:** Thanks for your insightful suggestion. In our revised paper, we add finer grained breakdown time and token cost statistics of MGRAG’s each parts in R-Table 7 (Table 8). In the experiment, we use the Qwen2.5-VL-7B-Instruct as the LLM backbone, aligning with our main experiment. We briefly analyze each aspect as follows:

- **Token Cost:** According to R-Table 7 (Table 8), our LLM entity and relation extraction is the current token cost bottleneck of our MGRAG model. However, according to R-Table 13 (Table 6) our token cost is still much lower than the current most efficient graph-based RAG model HippoRAG2 [25] on the MMQA dataset,



**R-Figure 6: End-to-end time cost comparison of MGRAG on different single GPU resources on MMQA dataset with Qwen2.5-VL-7B-Instruct backbone.**

and also not a huge additional cost compared to Similarity-based RAG models. Based on current GPT-5.2 pricing, the additional 3,500 tokens per query used by MGRAG compared to VanillaRAG result in an extra cost of only \$0.035 per query. This is a small price for the significant performance improvement that MGRAG achieves on long-hop queries.

- **Time Latency:** In R-Table 7 (Table 8), we provide a breakdown time cost of the multimodal KG construction (MMKGC), coarse grained retrieval (Coarse), semantic path-based subgraph matching (Matching), and the final LLM QA (QA) component of our MGRAG. Specifically, similar to the token cost, MGRAG’s LLM entity and relation extraction is also the current latency bottleneck. In average, the KG construction step takes 71.2% of the total time of MGRAG, while the subgraph matching and LLM QA take 0.1% and 19.1% of the total time, respectively. All these time cost statistics indicate that the repeated LLM calls for entity and relation extraction is the key bottleneck of our MGRAG model, and also a future research direction for improving the efficiency of graph-based RAG models. However, according to R-Table 13 (Table 6) our time latency is much lower than other graph-based RAG models such as GraphRAG [15] and LightRAG [23], and also not a huge additional cost compared to Similarity-based RAG models.
- **GPU Usage:** As shown in R-Figure 6, we evaluated the end-to-end time cost of MGRAG on different single GPUs (3090 24GB, A100 40GB, and A100 80GB) using the MMQA dataset and Qwen2.5-VL-7B-Instruct backbone. We do not report effectiveness since they are same in different GPUs. The results indicate that MGRAG can operate efficiently on mainstream hardware, and upgrading to more powerful GPUs yields only marginal reductions in total computation time (4.4 h vs. 4.1h). This supports the practical scalability of our method for real-world deployments with standard hardware.

**R5W2, R5D2:** *Limited discussion on the greedy algorithm: Are there queries or datasets for which this might fail? Are there alternate approaches for certain datasets?*

**Reply to R5W2, R5D2:** Thank you for your insightful suggestion. In our revised paper, we compare MGRAG and VanillaRAG’s [20] performance on different hop queries from the MRAMG benchmark [61], and the result is listed in R-Table 8 (Table 9). According to the results, MGRAG’s advantage grows with query complexity: on 1–3 hop queries, MGRAG only outperforms VanillaRAG by 5.1% on accuracy while costing 0.25 minutes more per query, as our path-based semantic subgraph matching is designed for long-range reasoning and provides limited benefit on simple queries. However, on 4–6 hop queries, MGRAG significantly outperforms VanillaRAG by 27.4% on accuracy, as our mechanism effectively captures long-range reasoning paths in the graph. Therefore, for simple few-hop queries where efficiency is prioritized, simpler methods such as VanillaRAG can be a more practical choice, while for complex multi-hop queries, MGRAG offers substantial gains that justify its additional cost.

In practice, we recommend users adopt VanillaRAG for datasets dominated by simple 1 hop queries, and switch to MGRAG when queries require multi-hop reasoning (2+ hops), where its path-based matching yields the most significant improvements. We have added the related discussion in Section 4.2 of our revised paper.

# MGRAG: Semantic Subgraph Matching and Graph-Aware Caching for Multimodal Retrieval-Augmented Generation

Yubo Wang

The Hong Kong University of Science  
and Technology  
Hong Kong SAR, China  
ywangnx@connect.ust.hk

Haoyang Li

The Hong Kong Polytechnic  
University  
Hong Kong SAR, China  
haoyang-comp.li@polyu.edu.hk

Lei Chen

The Hong Kong University of Science  
and Technology (Guangzhou)  
Guangzhou, China  
leichen@hkust-gz.edu.cn

## ABSTRACT

Multimodal Retrieval-Augmented Generation (RAG) enhances large language models (LLMs) by retrieving relevant text and image content to support query answering. Among RAG systems, the Graph-based RAG systems retrieve the fine-grained entity-level information, leading to promising effectiveness; however, they can still suffer from the following limitations: (1) *query-agnostic Knowledge Graph (KG) construction*, as the query-related visual context can be overwhelmed by query-irrelevant noises during the graph construction step; (2) *inflexible graph match*, which relies on rigid topological matching from query to graph and apply fixed retrieval size. (3) *event-agnostic KV re-computation*, which fails to preserve the event-level semantic structure in multimodal contexts when applying KV cache. To address these issues, we propose a multimodal graph-based RAG framework, namely MGRAG. Firstly, MGRAG constructs a query-specific KG in a top-down, iterative manner, guided by chunks’ semantic similarity to the query. Secondly, it introduces the path-based semantic subgraph matching problem, which we prove to be NP-hard, and we design an efficient greedy algorithm to solve it, enabling flexible, semantics-aware graph retrieval. Finally, MGRAG employs an event-aware KV caching mechanism that leverages graph topology to selectively recompute tokens critical to query-related events in multimodal contexts, making the KV caching better reflect the importance of each graph component. Experiments on seven real-world multimodal QA datasets show that MGRAG achieves better performance than state-of-the-art RAG and KV caching baselines.

## PVLDB Reference Format:

Yubo Wang, Haoyang Li, and Lei Chen. MGRAG: Semantic Subgraph Matching and Graph-Aware Caching for Multimodal Retrieval-Augmented Generation. PVLDB, 14(1): XXX-XXX, 2020.  
doi:XX.XX/XXX.XX

## PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/Wyb0627/MGRAG>.

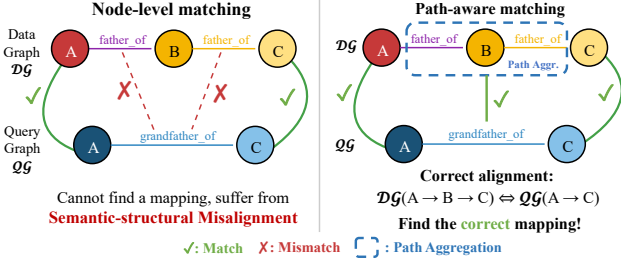
## 1 INTRODUCTION

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.  
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX

Answering complex queries over large, heterogeneous multimodal document corpora is a central challenge in data management [47, 48, 50, 54, 56]. Given a query and a corpus containing textual and image documents, the system is expected to retrieve the minimal set of fine-grained, entity-level evidence that is both sufficient and precise for answering the query, and serve the retrieved context to a generative model efficiently, especially when many queries share overlapping evidence. The first step is fundamentally a query-retrieval problem over structured or semi-structured data, closely related to subgraph matching [31, 67] and query processing over knowledge graphs [33] studied extensively in the database community. The second is a cache-management problem for intermediate computation states, akin to buffer and view management in query processing systems [34, 60]. While modern large language models (LLMs) such as GPT4 [3] and Qwen2.5 [7] serve as powerful generation backends, the quality of their answers is ultimately bounded by the quality of the retrieved evidence [18, 37, 39, 66]. This motivates the design of more principled retrieval and caching mechanisms as core data-management contributions, with LLMs positioned as the downstream consumer of the retrieved results.

Multimodal retrieval-augmented generation (RAG) [15, 19, 20, 23–25] provides a general framework for this task: an indexing module organizes the corpus, a retrieval module selects partial content across modalities, and an LLM generates the final answer. A key data-management challenge is to retrieve information that is both fine-grained enough to support multi-hop reasoning and compact enough to avoid the noise and latency penalties of overly lengthy contexts [32, 41, 44, 66]. Depending on their indexing and retrieval strategies, current multimodal RAG systems fall into two categories. *Similarity-based RAG* systems [20, 43, 63] encode multimodal data into a shared vector space using encoders such as CLIP [46] and retrieve the top- $k$  most similar chunks. However, representing each chunk as a single vector obscures fine-grained entity-level information, leading to coarse-grained retrieval. To achieve entity-level granularity, *Graph-based RAG* systems [8, 15, 23–26, 64] use LLMs to extract entities and relations from multimodal documents, organize them as nodes and edges in a knowledge graph (KG), and retrieve relevant graph components at query time. However, from a data-management perspective, existing graph-based RAG approaches still face three fundamental challenges in their indexing, retrieval, and caching stages.

**Challenge 1: Query-agnostic KG construction.** Existing graph-based approaches [15, 23, 25] construct a single, corpus-wide KG offline by extracting all entities and relations from every document, without considering the focus of any specific query. This is analogous to materializing a universal view without knowing the



**Figure 1: A motivating example of our path-based semantic subgraph matching.** The query’s `grandfather_of` edge matches a two-hop `father_of` path in the data graph. Node-wise matching (left) misses this alignment, while our method (right) aggregates paths to align them.

query workload: query-relevant entities, especially fine-grained visual events extracted from images, are overwhelmed by irrelevant noise, and information critical for a specific query may never be extracted at all. The challenge is to design an adaptive, query-driven graph construction strategy that incrementally builds a compact, query-specific KG from the most relevant documents, balancing construction cost against retrieval recall.

**Challenge 2: Inflexible graph matching.** Once a KG is available, the retrieval step should identify the subgraph that best answers the query. Current approaches either retrieve individual nodes or triples by vector similarity with a fixed result size  $k$  [23], or apply rigid neighborhood expansion [25], both of which ignore path-level semantic equivalences. As shown in Figure 1, consider a data graph  $\mathcal{DG}$  containing the two-hop path  $A \xrightarrow{\text{father\_of}} B \xrightarrow{\text{father\_of}} C$ , and a query graph  $\mathcal{QG}$  containing the single edge  $A \xrightarrow{\text{grandfather\_of}} C$ . Traditional node-level matching fails because no single edge in  $\mathcal{DG}$  corresponds to `grandfather_of`, resulting in an inflexible graph matching. This kind of mismatch arises frequently in multimodal KGs, where a single query edge about a visual event may correspond to a multi-hop path through fine-grained entities extracted from images.

**Challenge 3: Event-agnostic KV cache management.** In multi-user serving scenarios, current RAG serving systems [10, 34, 42, 60, 69] cache the key-value (KV) states of retrieved contexts to accelerate inference for subsequent queries that reuse the same chunks. Existing methods, such as CacheBlend [60], decide which tokens to selectively recompute based solely on token-level cross-attention scores. However, in multimodal contexts, particularly when processing images, each image is represented as a sequence of discrete tokens, fragmenting holistic visual events into isolated segments. Token-level scoring fails to capture the event-level structural and semantic relationships among these tokens, which was naturally encoded in the KG topology.

**Our approach.** To address these three data-management challenges, we propose MGRAG, a graph-based multimodal RAG framework with three core stages. *First*, MGRAG uses dense encoders to rank multimodal chunks by semantic relevance to the query, then

incrementally constructs a query-specific multimodal KG by extracting entities and relations with an LLM in a query-aware, top-down manner. Drawing on the database principle of lazy view materialization, MGRAG constructs only the graph fragment needed for the current query, with an early-termination criterion driven by subgraph matching, thereby avoiding the noise, information loss, and prohibitive indexing cost of corpus-wide, query-agnostic graph construction. *Second*, to enable adaptive and semantically flexible graph retrieval, we formulate a Path-based Semantic Subgraph Matching problem that generalizes classical subgraph isomorphism [13, 21] by relaxing topological constraints with path-level semantic aggregation. We prove this problem is NP-hard and design an efficient greedy algorithm based on semantic path growing and VF2-based isomorphism enumeration [13], whose practical cost is modest on the small, query-specific graphs constructed by MGRAG. *Third*, MGRAG introduces an event-aware KV caching mechanism that incorporates four graph centrality measures: degree, betweenness, eigenvector, and closeness. MGRAG utilizes these features to guide selective re-computation of cached states, bridging the gap between token-level attention scores and event-level structural importance and preserving critical semantic relationships during inference.

We summarize our contributions, which center on graph construction, query retrieval, and cache management:

- We propose MGRAG, a multimodal graph-based RAG system that incrementally builds query-specific KGs on demand via a lazy, top-down construction strategy, avoiding the cost and quality drawbacks of corpus-wide graph indexing.
- We formulate graph retrieval as a new path-based semantic subgraph matching problem, prove it NP-hard, and design an efficient greedy algorithm to solve it.
- We introduce a cache re-computation policy guided by graph centrality, enabling the caching mechanism to preserve semantic coherence in multimodal contexts.
- Experiments on seven real-world multimodal QA datasets demonstrate that MGRAG achieves superior effectiveness and efficiency compared to state-of-the-art RAG, subgraph matching, and KV caching baselines.

## 2 PRELIMINARY AND RELATED WORKS

This section reviews related work on three core aspects of multimodal RAG data management: multimodal RAG pipelines (Section 2.1), subgraph matching (Section 2.2), and KV cache management (Section 2.3). Key notations are summarized in Table 1.

### 2.1 Multimodal RAG

Although LLMs excel at multimodal tasks, they often lack up-to-date or domain-specific knowledge and struggle to recall long-tail facts, leading to hallucinated responses [37, 39, 66]. Multimodal RAG systems [2, 4, 15, 20, 23–25, 43, 49, 55, 62, 65, 68] address these gaps by retrieving up-to-date, relevant multimodal content, such as text and images, from large external sources. Formally, given a user query  $q$ , a multimodal document corpus  $\mathcal{D} = \mathcal{D}_{\text{txt}} \cup \mathcal{D}_{\text{img}}$  consisting of textual documents  $\mathcal{D}_{\text{txt}}$  and image documents  $\mathcal{D}_{\text{img}}$ , and an LLM with multimodal capabilities, a retrieval module  $\Omega(\cdot)$  first retrieves a set of query-relevant information contexts  $\mathcal{C}_q$  from the multimodal chunk set  $\mathcal{C}$ , which is generated from the original

corpus  $\mathcal{D}$  by an indexing module  $\Gamma$ :  $C_q = \Omega(C, q)$ ,  $C = \Gamma(\mathcal{D})$ . Next, the LLM generates an answer  $A_q$  conditioned on the query and the retrieved contexts  $C_q$ :  $\mathcal{A}_q = \text{LLM}(q, C_q)$ .

Depending on the technique, multimodal RAG approaches [4, 15, 20, 23–25, 43, 49, 55, 62, 65] fall into two categories: similarity-based and graph-based. In similarity-based systems [4, 20, 43, 49, 55, 65],  $\Gamma$  encodes queries and documents into a shared embedding space, and  $\Omega$  returns the top- $k$  most similar chunks or images. For example, VanillaRAG [20] encodes text chunks and images with pre-trained multimodal encoders (e.g., CLIP [46]) and retrieves by cosine similarity. While straightforward, this coarse-grained retrieval cannot capture fine-grained, entity-level semantics that span multiple chunks, nor does it provide explicit relational reasoning to the LLM about why each chunk was retrieved. On the another hand, graph-based systems [15, 23–25, 62] address this by having  $\Gamma$  construct a knowledge graph from multimodal documents and having  $\Omega$  retrieve structurally relevant subgraphs. GraphRAG [15] partitions the graph into communities and generates community-level summaries, but incurs high costs for community detection and summarization. HippoRAG [24] scores nodes via Personalized PageRank and retrieves chunks associated with top-scoring nodes, reducing context length. HippoRAG2 [25] further improves graph connectivity by indexing entire passages as nodes.

Despite recent advances, existing graph-based systems have three main limitations: (1) query-agnostic graph construction leads to high indexing costs and noisy, less accurate retrieval; (2) retrieval is limited to node or edge matching, missing multi-hop path semantics; (3) a fixed retrieval budget  $k$  cannot flexibly support queries needing variable evidence.

## 2.2 Subgraph Matching

To adaptively retrieve query-relevant information from the external KG, subgraph matching methods can identify the most relevant subgraphs that align with the query graph, directly affecting both the effectiveness and efficiency of query retrieval in the RAG data-management pipeline. Classical methods can be grouped into three categories: exact solvers [13, 28], approximate graph-matching algorithms [11, 17, 35, 36], and semantic graph-alignment methods [52, 53, 58].

Exact solvers [13, 28] enumerate all subgraphs of  $\mathcal{KG}$  isomorphic to  $Q_q$ . VF2 [13] uses depth-first backtracking with feasibility pruning on degrees, neighborhoods, and labels, while ISMAGS [28] extends it with symmetry-aware search to avoid redundant embeddings. However, exact solvers require strict topological isomorphism, making them unable to address the inflexible graph matching issue in Figure 1. Also, approximate graph-matching algorithms relax isomorphism constraints into optimization over soft correspondence matrices. SM [35] uses eigen-decomposition of an affinity matrix for soft scores, RRWM [11] interprets matching as a reweighted random walk with Sinkhorn normalization, VJ [17] expands seeded correspondences via bipartite matching, and IPFP [36] solves a quadratic assignment relaxation via integer-projected fixed-point iteration. These methods tolerate structural noise but still operate at individual node or edge level without aggregating path-level semantics. Lastly, semantic graph-alignment methods operate in embedding spaces derived from node/edge

**Table 1: Summary of important notations.**

| Notations                                       | Meanings                                                      |
|-------------------------------------------------|---------------------------------------------------------------|
| $q$                                             | The input user query.                                         |
| $\mathcal{D}$                                   | The multimodal document corpus.                               |
| $\Omega$                                        | The multimodal retrieval module.                              |
| $\mathcal{D}_{txt}, \mathcal{D}_{img}$          | The set of textual and image documents in $\mathcal{D}$ .     |
| $\Gamma, \Omega$                                | The indexing and retrieval module.                            |
| $C$                                             | The set of multimodal chunks generated from $\mathcal{D}$ .   |
| $\mathcal{T}$                                   | The set of text chunks split from $\mathcal{D}_{txt}$ .       |
| $c$                                             | A multimodal chunk (text or image).                           |
| $\vec{h}_*$                                     | The vector encoding of the respective component.              |
| $C_q^{rank}$                                    | The coarse-grained ranked chunks regarding query $q$ .        |
| $\mathcal{KG}_q(\mathcal{V}_q, \mathcal{E}_q)$  | The query-specific multimodal KG.                             |
| $\mathcal{V}_q, \mathcal{E}_q$                  | The set of entities and relations in $\mathcal{KG}_q$ .       |
| $Q_q(\hat{\mathcal{V}}_q, \hat{\mathcal{E}}_q)$ | The query graph constructed from query $q$ .                  |
| $\hat{\mathcal{V}}_q$                           | Entities (including placeholders) in the query graph.         |
| $\hat{\mathcal{E}}_q$                           | Relations in the query graph.                                 |
| $\mathcal{R}$                                   | The candidate matched subgraph set                            |
| $res$                                           | The final matched subgraph.                                   |
| $S_{res}$                                       | Matched subgraph $\mathcal{R}$ 's serialized string sequence. |
| $\oplus(\cdot)$                                 | The path aggregation function.                                |
| $\theta$                                        | Semantic similarity threshold for subgraph matching.          |
| $\phi(t)$                                       | The KV recompute score for token $t$ .                        |
| $GF_t^q$                                        | The query specific graph topological score for token $t$ .    |

attributes. SLOAlign [52] jointly refines graph structure and computes an optimal-transport alignment between attributed graphs, while FGWAlign [53] employs fused Gromov-Wasserstein distances to jointly model attribute and topological similarity. Despite leveraging semantic embeddings, these methods still match at the node or edge level without modeling path-level, query-aware semantics.

## 2.3 KV Cache Management

To mitigate the high computational cost of autoregressive decoding in LLMs, key-value (KV) caching is widely adopted during inference. The core idea is to reuse previously computed key and value vectors in the attention mechanism, avoiding redundant recomputation across decoding steps [38]. Formally, consider an LLM with  $L$  transformer layers. At each layer  $\ell$ , for an input token sequence  $\mathbf{x} = [x_1, \dots, x_n]$ , the self-attention mechanism computes query  $\mathbf{Q}^\ell$ , key  $\mathbf{K}^\ell$ , and value  $\mathbf{V}^\ell$  matrices as:  $\mathbf{Q}^\ell = \mathbf{X}\mathbf{W}_Q^\ell$ ,  $\mathbf{K}^\ell = \mathbf{X}\mathbf{W}_K^\ell$ ,  $\mathbf{V}^\ell = \mathbf{X}\mathbf{W}_V^\ell$ , where  $\mathbf{X}$  is the token embedding matrix, and  $\mathbf{W}_Q^\ell, \mathbf{W}_K^\ell, \mathbf{W}_V^\ell$  are learnable projection matrices. The attention output is then:

$$\text{Attention}(\mathbf{Q}^\ell, \mathbf{K}^\ell, \mathbf{V}^\ell) = \text{softmax}\left(\frac{\mathbf{Q}^\ell(\mathbf{K}^\ell)^\top}{\sqrt{d_k}}\right)\mathbf{V}^\ell. \quad (1)$$

During autoregressive generation, previously computed key/value (KV) matrices for tokens  $x_1, \dots, x_n$  can be cached and reused, since they depend only on past tokens. This caching speeds up inference. However, in Retrieval-Augmented Generation (RAG), the retrieved context  $C_q$  and its integration (e.g., via positional encoding or cross-attention) can vary per query. Thus, blindly reusing cached KV for identical context chunks across queries may harm generation quality, as their representation can differ by query. Proper cache management is necessary.

**DEFINITION 1 (KV CACHE MANAGEMENT).** Given a user query  $q$ , a set of multimodal contexts  $C_q$ , and a transformer-based LLM, KV cache management accelerates autoregressive decoding by reusing previously computed key-value (KV) states for tokens in  $C_q \cup \{q\}$ , while selectively recomputing KV states for a subset of tokens critical to generation quality. Formally, let  $\mathcal{T} = \text{Tokenize}(q \parallel C_q)$  denote the full input token sequence, with  $\parallel$  indicating concatenation. For each token  $t \in \mathcal{T}$ , the system computes a recompute score  $\phi(t)$  reflecting its importance. At each transformer layer, only the top- $b\%$  tokens with highest  $\phi(t)$  values are recomputed, while the KV states of the remaining tokens are reused from cache:

$$KV(t) = \begin{cases} \text{Recompute}(t), & \text{if } \phi(t) \text{ is in top } b\% \\ \text{Cache}(t), & \text{otherwise} \end{cases} \quad (2)$$

where  $\text{Cache}(t)$  denotes the cached KV state from a previous inference, and  $\text{Recompute}(t)$  denotes freshly computed KV projections.

Current KV cache mechanisms are classified as Position Dependent Caching and Position Independent Caching. Position dependent approaches, such as vLLM [34], reuse KV matrices for tokens at the same positions, which improves efficiency for repeated prompts but provides limited acceleration due to short shared subsequences. Position independent approaches [5, 60] reuse KV caches based on token importance, such as feature distances [5] or attention scores [60], allowing broader reuse and greater speedup. However, these methods do not consider the structural relationships among tokens, like events in image data, which restricts their effectiveness.

### 3 METHODOLOGY

In this section, we first introduce the overall MGRAG framework. We then detail its three core stages: Coarse-grain Ranking, Fine-grain Retrieval, and Graph-Aware KV Cache. Finally, we will deliver the complexity analysis of MGRAG.

#### 3.1 Framework Overview

To address *query-agnostic KG construction, inflexible graph matching, and event-agnostic KV re-computation issues of current multimodal RAG systems*. We propose MGRAG, as shown in Figure 2, MGRAG consists of 3 stages, i.e., Coarse-grain Ranking, Fine-grain Retrieval, and Graph-Aware KV Caching.

**Step 1: Coarse-grain Ranking.** MGRAG first encodes source multimodal documents in  $\mathcal{D} = \{\mathcal{D}_{txt}, \mathcal{D}_{img}\}$  to prepare for retrieval. Text documents in  $\mathcal{D}_{txt}$  are split into overlapping chunks and encoded, while image captions  $C$  are used as vector representations for images in  $\mathcal{D}_{img}$ , forming the multimodal chunks  $C$ . MGRAG then ranks multimodal chunks  $C$  regarding their relevance to the query  $q$ , obtaining ranked chunks  $C_q^{\text{rank}}$ .

**Step 2: Fine-grain Retrieval.** In this step, MGRAG iterates the ranked multimodal chunks  $C$  top-down to construct or enrich the query  $q$  specific multimodal KG  $\mathcal{KG}_q$ . The iteration terminates when a semantically matched subgraph is found between the query graph  $\mathcal{QG}_q$  and the multimodal KG  $\mathcal{KG}_q$ , or when the iteration limit  $\mu$  is reached. We formalize subgraph matching as a *Path-based Semantic Subgraph Matching* problem, which we prove to be NP-hard. In this formulation, a path  $P$  in the query graph  $\mathcal{QG}_q$  is mapped to a subgraph  $r$  in the multimodal KG  $\mathcal{KG}_q$ . To solve this problem, we design a greedy algorithm.

**Table 2: The scoring function of KV caching methods, where Prefix denotes the tokens in the input prefix,  $KV^{\text{full}}[t]$  and  $KV[t]$  denotes the fully computed, and previously cached KV matrices of token  $t$ , respectively.**

| Method          | Feature                   | Scoring Function $\phi(t)$                                     |
|-----------------|---------------------------|----------------------------------------------------------------|
| vLLM [34]       | Position Dependent        | $1[t \in \text{Prefix}]$                                       |
| CacheBlend [60] | Position Independent      | $\ KV[t] - KV^{\text{full}}[t]\ _2$                            |
| MGRAG           | Graph Structure Dependent | $\sum_{j=1}^{ q } a_{tj} + \sum_{k=1}^{ S_r } a_{tk} + GF_t^q$ |

**Step 3: Graph-Aware KV Cache.** Here, MGRAG selectively recomputes the KV cache of each token  $t$  based on a cache recompute score  $\phi(t)$ , which comprises three components: (1) the cross-attention score between token  $t$  and the query  $q$ , (2) the cross-attention score between token  $t$  and the serialized graph sequence  $S_r$ , and (3) the graph feature score  $GF_t^q$ . To capture topological properties in small graphs,  $GF_t^q$  incorporates four key graph topological measures of the triplet fact on graph that associated with token  $t$ : degree, betweenness, closeness, and eigenvector centrality. MGRAG will recompute the KV matrix for  $b\%$  of the tokens in each LLM layer.

At last, MGRAG would input the matched subgraph sequence  $S_r$ , the multimodal chunks  $C_q$  used to construct the multimodal KG  $\mathcal{KG}_q$ , and query  $q$  together into the LLM for final query answering.

#### 3.2 Step 1: Coarse-grain Ranking

In this subsection, we introduce the Coarse-grain Ranking step of MGRAG. We first encode the source multimodal document set  $\mathcal{D}$  for retrieval, where  $\mathcal{D} = \mathcal{D}_{txt} \cup \mathcal{D}_{img}$ , with text document set  $\mathcal{D}_{txt}$  and image set  $\mathcal{D}_{img}$ . Specifically, for textual documents in  $\mathcal{D}_{txt}$ , we will split it into overlapping text chunks  $\mathcal{T}$ , with a fixed chunk token length  $l_c$ , and an overlapping token length  $l_o$ :  $\mathcal{T} = \text{Split}(\mathcal{D}_{txt}, l_c, l_o)$ , where we set  $l_c$  and  $l_o$  to 256 and 32, respectively, to align the best empirically finding in existing work [24, 25].

We combine all text chunks  $\mathcal{T}$  and image documents  $\mathcal{D}_{img}$  to form the multimodal chunk set  $C = \mathcal{T} \cup \mathcal{D}_{img}$ . Each chunk  $c \in C$  is encoded with a pretrained encoder  $\mathcal{M}$  to obtain its embedding  $\vec{h}_c \in \mathbb{R}^d$  for coarse-grained ranking:  $\vec{h}_c = \mathcal{M}(c)$ , where  $d$  is the embedding dimension. Following [22], we represent images using their captions so that both modalities use the same text encoder. We use bge-base-en-v1.5 [58] as our encoder. If  $c$  is an image ( $c \in \mathcal{D}_{img}$ ), we encode its caption, either provided by the dataset or generated by an LLM, as its embedding.

Given a query  $q$  that may consist of both text  $q_{txt}$  and image  $q_{img}$  modalities, our goal is to rank the most relevant multimodal chunks in  $C = \{c_1, c_2, \dots, c_{|C|}\}$ . Each chunk  $c_j$  is text or image chunk. For a given query  $q$ , if it contains only a single modality (either text or image), we use the corresponding encoder to obtain the query embedding  $\vec{h}_q$ . If  $q$  consists of both text and image modalities, we compute its representation  $\vec{h}_q$  by averaging the embeddings of its textual component and the caption  $q_{cap}$  of the querying image:

$$\vec{h}_q = \frac{\mathcal{M}(q_{txt}) + \mathcal{M}(q_{cap})}{2}. \quad (3)$$

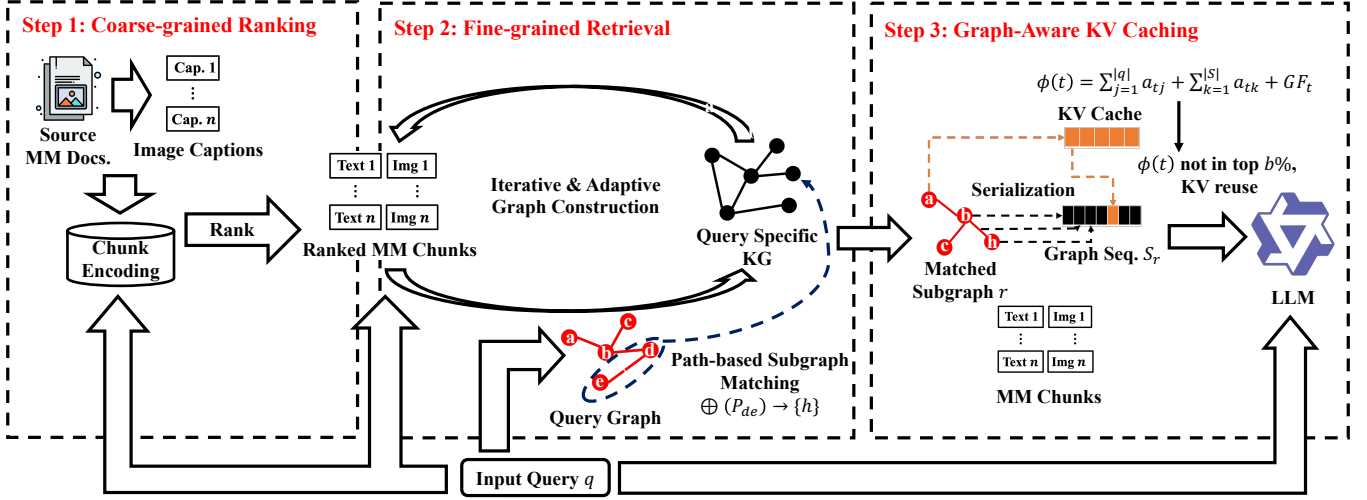


Figure 2: Our MGRAG consists of 3 steps. In Step 1, we rank the multimodal chunks regarding their relevance to the query. In Step 2, we iteratively construct a query-specific KG with the ranked multimodal chunks in a top-down manner. The iterations stop when the query’s semantically aligned subgraph is found in KG via a Path-based Subgraph Matching mechanism. In Step 3, We selectively reuse the LLM’s KV cache from Step 2, scoring each token  $t$  by its topological importance  $GF_t^q$  in KG and cross-attention weights  $a$ , and only recompute tokens with recompute score  $\phi(t)$  in top  $b\%$ .

Next, we evaluate the distance between the query and each chunk using the Euclidean distance in the embedding space:

$$\text{distance}(\vec{h}_q, \vec{h}_c) = \|\vec{h}_q - \vec{h}_c\|_2, \quad (4)$$

where a smaller distance indicates a higher degree of semantic similarity between the query and the chunk. This metric enables an effective comparison across modalities by leveraging the strengths of the multimodal encoder.

With all distance scores computed, we rank the entire set of candidate chunks for each query by sorting them in ascending order of their Euclidean distance to the query representation, this rank results  $C_q^{\text{rank}}$  of coarse-grained chunks  $C$  for the query  $q$  is:

$$C_q^{\text{rank}} = \left\{ c_i \in C \mid i \in \text{argsort}_{\uparrow} \left( \left[ \text{distance}(\vec{h}_q, \vec{h}_{c_j}) \right]_{j=1}^{|C|} \right) \right\}. \quad (5)$$

### 3.3 Step 2: Fine-grain Retrieval

In this subsection, we introduce the Fine-grain Retrieval step of MGRAG. Here, we iteratively select chunks  $c$  from the query  $q$ ’s ranked chunks  $C_q^{\text{rank}}$  in descending order of similarity to the query. Each selected chunk is used to extract entities and relations, thereby incrementally enriching the query-specific multimodal KG  $\mathcal{KG}_q$ . The iteration terminates when our *Path-based Semantic Subgraph Matching* algorithm identifies a matched subgraph  $r \subseteq \mathcal{KG}_q$ , or reach the maximum iteration  $\mu$ .

Firstly, we construct the query graph  $\mathcal{QG}_q(\hat{\mathcal{V}}_q, \hat{\mathcal{E}}_q)$  regarding the query  $q$ . Here, we extract key entities  $\hat{\mathcal{V}}_q$  and relations  $\hat{\mathcal{E}}_q$  from the query similarly to Equation (6). When extracting entities and relations, as shown in Figure 3, to represent the key missing entities and relations in query, which are usually highly related to the query answer, we ask the LLM to use placeholders that are semantically close to those entities or relations, such as *Person*, or *Year*, to represent them, and serve as entities or relations on query graph. These placeholders are later used for calculating semantic

similarity with components on the multimodal KG, to retrieve the answer-related information.

Next, we construct or enrich the query-specific KG  $\mathcal{KG}_q$ . In each iteration, we first apply LLM (e.g., Qwen-2.5-VL [7]) to extract entities  $\mathcal{V}_q^c$  and relations  $\mathcal{E}_q^c$  from each chunk  $c \in C_q^{\text{rank}}$  at the current iteration regarding query  $q$ :

$$\mathcal{V}_q^c, \mathcal{E}_q^c = \text{LLM}(c, q, p_{\text{extraction}}), \quad (6)$$

where  $\text{LLM}(\cdot)$  is the extraction model and  $p_{\text{extraction}}$  is the extraction prompt. In  $p_{\text{ext}}$ , we extract query-related entities and relations as key events from multimodal chunks, enabling query-aware KG construction and addressing the query-agnostic issue. The original multimodal chunks  $c$  are added to  $\mathcal{V}_q^c$  as passage nodes, yielding the final entity set  $\mathcal{V}_q$ . We then connect the passage node  $c$  to entity  $v$  extracted from it with pseudo relation  $e(c, v)$ , with relation *consists\_in*, obtaining the final relation set  $\mathcal{E}_q$ :

$$\mathcal{V}_q = \bigcup_{c \in C_q^{\text{rank}}} \mathcal{V}_q^c \cup \{c\}, \quad \mathcal{E}_q = \bigcup_{c \in C_q^{\text{rank}}} (\mathcal{E}_q^c \cup \bigcup_{v \in \mathcal{V}_q^c} f(c, v)), \quad (7)$$

where  $f(c, v)$  returns  $\{e(c, \text{consists\_in}, v)\}$  if  $c$  satisfies the *consists\_in* condition, and  $\emptyset$  otherwise. With the entity set  $\mathcal{V}_q$  and relation set  $\mathcal{E}_q$ , we are able to construct the multimodal KG  $\mathcal{KG}_q(\mathcal{V}_q, \mathcal{E}_q)$  specific to query  $q$ . Then, given an edge  $e(u, r, v) \in \mathcal{E}_q$ , where  $u$  and  $v$  are entities and  $r$  is the relation, we obtain their representations using a text encoder  $\mathcal{M}$  (e.g., BGE [58]) as follows:

$$\vec{h}_u = \mathcal{M}(u), \quad \vec{h}_v = \mathcal{M}(v), \quad \vec{h}_r = \mathcal{M}(r). \quad (8)$$

Also, given two entities  $u$  and  $v$ , there may exist multiple paths from  $u$  to  $v$ , denoted as  $\mathcal{P}(u, v)$ . A single path  $P_i(u, v) \in \mathcal{P}(u, v)$  consisting of  $k$  consecutive triples is defined as:

$$P_i(u, v) = (e_1(u, r_1, z_1), e_2(z_1, r_2, z_2), \dots, e_k(z_{k-1}, r_k, v)),$$

where  $z_1, \dots, z_{k-1}$  are intermediate entities,  $r_1, \dots, r_k$  are the corresponding relations, and the path length is  $|P(u, v)| = k$ . We define

Query: The Irandhir Santos film whose poster has people doing martial arts was released in what year?

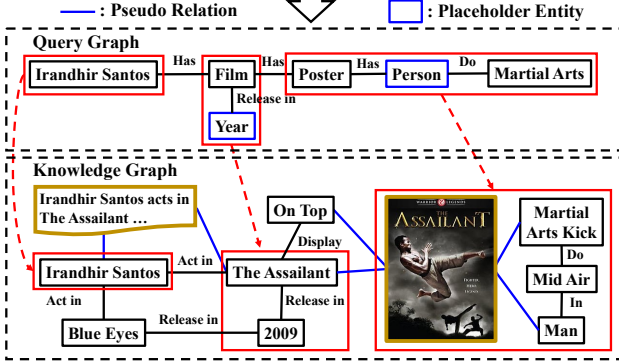


Figure 3: An example of our path-based semantic subgraph matching. The **blue edges** denote pseudo relations that connect **multimodal chunks** (passage nodes) to entities extracted from them. The **blue boxes** denote placeholder entities, representing query unknown knowledge.

the embedding of path  $P(u, v)$  as the aggregation over its relation embeddings:

$$\vec{h}_{P(u,v)} = \bigoplus_{j=1}^{|P(u,v)|} \vec{h}_{r_j}, \quad (9)$$

where  $\vec{h}_{r_j}$  is the semantic embedding of relation  $r_i$ , and  $\bigoplus$  is a customized aggregation function, such as mean pooling.

After the graph construction or enrichment in each iteration, a straightforward approach is to apply a graph matching algorithm based on each node/relation’s pair wise semantic similarity. Concretely, given the query graph  $QG_q$ , if a matched subgraph can be identified in  $KG_q$ , we consider the available knowledge to be sufficient, terminate the iterative graph construction, and return the matched subgraph as the graph context to aid LLM query answering. However, this matching process faces two main challenges. First, as shown in Figure 3, semantically equivalent subgraphs may differ in topology, resulting in a semantic-structural mismatch that standard graph matching cannot resolve. Second, since MGRAG constructs a query-specific KG for each query, it is impractical to train a dedicated subgraph matching model for every KG. This limits the applicability of existing methods [31, 33, 67] in our setting. To address both challenges, we formulate this problem as the *Path-based Semantic Subgraph Matching Problem*.

**DEFINITION 2 (PATH-BASED SEMANTIC SUBGRAPH MATCHING PROBLEM).** Given a query graph  $QG_q(\hat{\mathcal{V}}_q, \hat{\mathcal{E}}_q)$ , a multimodal KG  $KG_q(\mathcal{V}_q, \mathcal{E}_q)$ , and a similarity threshold  $\theta \in [0, 1]$ , the *Path-based Semantic Subgraph Matching Problem* is to find an optimal matched subgraph  $SG(\mathcal{V}_q, \mathcal{E}_q) \subseteq KG_q$ , induced by a mapping  $I : \hat{\mathcal{V}}_q \rightarrow \mathcal{V}_q$ , to maximize the overall similarity:

$$\begin{aligned} \max \text{Sim}(SG) &= \sum_{e(u,r,v) \in \hat{\mathcal{E}}_q} \max_{P \in \mathcal{P}(I(u), I(v))} \cos(\vec{h}_r, \vec{h}_P) \quad (10) \\ \text{s.t.} \quad &\begin{cases} \cos(\vec{h}_u, \vec{h}_{I(u)}) > \theta, \cos(\vec{h}_v, \vec{h}_{I(v)}) > \theta, & \forall u, v \in \hat{\mathcal{V}}_q \\ \max_{P \in \mathcal{P}(I(u), I(v))} \cos(\vec{h}_r, \vec{h}_P) > \theta, & \forall \hat{e}(u, \hat{r}, v) \in \hat{\mathcal{E}}_q \end{cases} \end{aligned}$$

#### Algorithm 1: Path-based Semantic Subgraph Matching

**Input:** Query graph  $QG_q(\hat{\mathcal{V}}_q, \hat{\mathcal{E}}_q)$ , multimodal KG  $KG_q(\mathcal{V}_q, \mathcal{E}_q)$ , threshold  $\theta$

**Output:** Highest-scoring matched subgraph  $r \subseteq KG_q$  or  $\emptyset$

```

1  $\mathcal{R} \leftarrow \emptyset$ ;
2  $\mathcal{I} \leftarrow \text{SUBGRAPHISOMORPHISM}(QG_q, KG_q, v)$ ;
3 for each mapping  $I \in \mathcal{I}$  do
4    $\text{valid} \leftarrow \text{true}, s \leftarrow 0$ ;
5    $\mathcal{V}_{\text{sub}} \leftarrow \{I(x) \mid x \in \hat{\mathcal{V}}_q\}$ ;
6    $\mathcal{E}_{\text{sub}} \leftarrow \emptyset$ ;
7   for each  $\hat{e}(u, \hat{r}, v) \in \hat{\mathcal{E}}_q$  do
8      $P \leftarrow \text{GREEDYPATHSEARCH}(I(u), I(v), \vec{h}_{e(u,r,v)}, KG_q, \theta)$ ;
9     if  $P = \emptyset$  then
10        $\text{valid} \leftarrow \text{false}$ ;
11       break;
12      $s \leftarrow s + \cos(\vec{h}_P, \vec{h}_{e(u,r,v)})$ ;
13      $\mathcal{E}_{\text{sub}} \leftarrow \mathcal{E}_{\text{sub}} \cup \{e \in P\}$ ;
14      $\mathcal{V}_{\text{sub}} \leftarrow \mathcal{V}_{\text{sub}} \cup \{\text{nodes in } P\}$ ;
15   if  $\text{valid}$  then
16      $s \leftarrow s/|\hat{\mathcal{E}}_q|$ ;
17      $\Lambda \leftarrow (\mathcal{V}_{\text{sub}}, \mathcal{E}_{\text{sub}})$ ;
18      $\mathcal{R} \leftarrow \mathcal{R} \cup \{(\Lambda, s)\}$ ;
19 return top-scoring subgraph  $r \in \mathcal{R}$ , or  $\emptyset$  if  $\mathcal{R} = \emptyset$ ;
```

where  $\vec{h}_P$  is the aggregated path embedding for  $P \in \mathcal{P}(I(u), I(v))$  as defined in Equation (9). If no such subgraph exists, return  $\emptyset$ .

**THEOREM 1.** The Path-based Semantic Subgraph Matching Problem is NP-hard.

**PROOF SKETCH.** We establish NP-hardness through a polynomial-time reduction from the Subgraph Isomorphism problem by constructing an instance with uniform embedding vectors and unit path lengths. Due to the space limitation, we put our proof at <https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf>.  $\square$

Since this problem is NP-hard, it is infeasible to obtain the optimal result in polynomial time. Therefore, to solve this problem, we propose an efficient greedy algorithm based on semantic path growing and subgraph isomorphism enumeration, as shown in Algorithm 1, which can find a subgraph within the multimodal KG that semantically matches the query graph, but not necessarily identical in graph topology. Given a query graph  $QG_q$ , we first select its anchor entity  $\gamma$  to the multimodal KG  $KG_q$ , specifically, we compute the pairwise cosine similarities between the embeddings of all entities in  $QG_q$  and  $KG_q$ , and select the pair  $(\gamma, v)$  with the highest similarity, where  $\gamma \in \hat{\mathcal{V}}_q$  and  $v \in \mathcal{V}_q$ . If the algorithm cannot find a valid match starting with  $\gamma$ , it will try other anchor entities until a valid match is found. After that, we consider  $\gamma$  as the anchor entity in the query graph and  $v$  as its candidate match entity in the multimodal KG, and calculate the *Path-based Subgraph Isomorphism*. In this paper, we substitute the VF2 [13] algorithm’s

exact matching mechanism with similarity-based matching. This similarity-based matching uses a similarity threshold  $\theta$  to calculate the Path-based Subgraph Isomorphism in the line 3 of Algorithm 1.

Then, for each subgraph isomorphism  $I : \mathcal{V}_q \rightarrow \mathcal{V}_g$  that satisfies  $I(\gamma) = v$ , we try to find a matched subgraph edge by edge. To begin with, for every relation  $\hat{e}(u, \hat{r}, v)$  in the relation set  $\hat{\mathcal{E}}_q$  of the query graph  $\mathcal{QG}_q$ , we grow a path in the multimodal KG  $\mathcal{KG}_q$  starting from entity  $u$ 's isomorphic node  $I(u)$  in the multimodal KG by greedily selecting entities  $x$  within the neighbor entity set  $\mathcal{N}(P)$  of the currently generated path  $P$  on the multimodal KG.

Then, we greedily expand the path  $P$  by evaluating the neighbors of the current end node  $x$  of path  $P$ . We select the neighbor entity  $y \in \mathcal{N}(x)$  and its connecting relation  $e(x, r, y) \in \mathcal{E}_q$  that maximize the cosine similarity between the newly aggregated path embedding and the edge embedding  $\vec{h}_{\hat{e}(u, \hat{r}, v)}$  of the query relation  $\hat{e}(u, \hat{r}, v)$ :

$$\arg \max_{y \in \mathcal{N}(x), e(x, r, y) \in \mathcal{E}_q} \cos(\vec{h}_P \oplus \vec{h}_{e(x, r, y)} \oplus \vec{h}_y, \vec{h}_{\hat{e}(u, \hat{r}, v)}) \quad (11)$$

where  $\oplus$  denotes the path aggregation function defined in Equation (9). Once the optimal relation  $e(x, r, y)$  are identified, we append them to the current path  $P$ :

$$P = P \circ e(x, r, y), \quad (12)$$

where  $\circ$  denotes the path concatenation function. The path expansion continues until the path reaches the target node  $I(v)$ , or terminates if no neighbor yields an aggregated cosine similarity above  $\theta$ . In our experiments, for efficiency, we set a maximum path length  $l_p$  to prevent excessively long paths.

The path expansion continues until no neighbor yields a cosine similarity above  $\theta$ . Then, denote  $\oplus$  as the path aggregation function, the aggregated embedding  $\vec{h}_P$  (obtained via Equation 9) of the resulting path  $P$  is compared against the query edge embedding  $\vec{h}_{\hat{e}(u, \hat{r}, v)}$ , and the edge  $e(x, r, y)$  is deemed semantically consistent only if this similarity also exceeds  $\theta$ . A mapping  $I$  is accepted only when all query edges pass this path-based validation. Then the matched subgraph  $\Lambda$ , with its entity set  $\mathcal{V}_{\text{sub}}$ , relation set  $\mathcal{E}_{\text{sub}}$ , and its average edge-wise semantic similarity score  $s$  are added to the result set  $\mathcal{R}$ . All valid matches in  $\mathcal{R}$  are induced subgraphs of  $\mathcal{KG}_q$  over  $\{I(u) \mid u \in \mathcal{V}_q\}$ , and among all of them, we return the one  $r \in \mathcal{R}$  with the highest semantic score  $s$ . If  $\mathcal{R} = \emptyset$ , we will extract entities and relations with Equation 6 from the next multimodal chunk in  $\mathcal{C}_q^{\text{rank}}$ , to further enrich the query specific multimodal KG  $\mathcal{KG}_q$ . This design effectively balances exact structural matching with flexible semantic reasoning, enabling robust and efficient subgraph retrieval in the multimodal KG.

Next, we prove our Path-based Semantic Subgraph Matching algorithm has an approximation ratio bounded by the similarity threshold  $\theta$  via  $\infty$ -Wasserstein Bottleneck [9].

**THEOREM 2.** *The Algorithms 1 achieve an approximation ratio bounded by the similarity threshold  $\theta$  for the Path-based Semantic Subgraph Matching Problem, if a valid matched subgraph is found.*

**PROOF SKETCH.** By framing the graph matching process as a constrained optimal transport problem, we show that the local thresholding in our greedy algorithm bounds the  $\infty$ -Wasserstein (bottleneck) distance and achieves  $\theta$ -approximation. Due to the

---

#### Algorithm 2: Greedy Path Search (GREEDYPATHSEARCH)

---

**Input:** Start node  $v_{\text{start}}$ , target node  $v_{\text{target}}$ , target joint embedding  $\vec{h}_{\text{target}}$ , data graph  $\mathcal{KG}_q(\mathcal{V}_q, \mathcal{E}_q)$ , similarity threshold  $\theta$

**Output:** Tuple (found,  $P$ ) where found is Boolean and  $P$  is the matched path

```

1  $P \leftarrow (v_{\text{start}}), \quad x \leftarrow v_{\text{start}};$ 
2 while  $x \neq v_{\text{target}}$  do
3    $\mathcal{N}_{\text{valid}} \leftarrow \emptyset;$ 
4   for each neighbor  $y \in \mathcal{N}(x)$  and edge  $e(x, r, y) \in \mathcal{E}_q$  do
5      $\vec{h}_{\text{tmp}} \leftarrow \vec{h}_P \oplus \vec{h}_{e(x, r, y)} \oplus \vec{h}_y;$ 
6     if  $\cos(\vec{h}_{\text{tmp}}, \vec{h}_{\text{target}}) > \theta$  then
7        $\mathcal{N}_{\text{valid}} \leftarrow \mathcal{N}_{\text{valid}} \cup \{(y, e(x, r, y), \cos(\vec{h}_{\text{tmp}}, \vec{h}_{\text{target}}))\};$ 
8   if  $\mathcal{N}_{\text{valid}} == \emptyset$  then
9     return (false,  $\emptyset$ );
10   $(e^*, \text{score}) \leftarrow \arg \max_{(e, s) \in \mathcal{N}_{\text{valid}}} s;$ 
11   $P \leftarrow P \circ e^*;$ 
12   $x \leftarrow y^*;$ 
13 if  $x == v_{\text{target}}$  then
14   return (true,  $P$ );
15 else
16   return (false,  $\emptyset$ );
```

---

space limitation, we put our proof at <https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf>.  $\square$

After obtaining the matched subgraph  $r$ , to convert the graph into an LLM-compatible string format, we serialize it into a string sequence  $S_r$  of triplet facts, with [SEP] as the split marker between different facts:

$$S_r = e(u_1, r_1, v_1) \parallel [\text{SEP}] \cdots [\text{SEP}] \parallel e(u_n, r_n, v_n), \quad (13)$$

where  $u_1 \cdots u_n, v_1 \cdots v_n \in \mathcal{V}_q$  denotes entity nodes in graph  $r$ ,  $e(u_1, r_1, v_1) \cdots e(u_n, r_n, v_n) \in \mathcal{E}_q$  denotes relation edges in graph  $r$ ,  $\parallel$  denotes the string concatenation operation.

In Step 2, the entities and relations of graph  $r$  are all extracted by the LLM to form the query-specific KG  $\mathcal{KG}_q$ , and will be input to the LLM again in graph sequence  $S_r$  for query answering. According to Figure 4, the **green token sequence** was input to the LLM for graph construction, and can be partially reused by tokens in the **blue graph sequence**  $S_r$ , as both of them are tokens of entities and relations in the KG. Hence, we propose a Graph-Aware KV Caching mechanism in Step 3, to better reflect the importance of these graph components' respective tokens, regarding the KG's topological structure.

### 3.4 Step 3: Graph-Aware KV Caching

In this subsection, we introduce the Graph-aware KV Caching mechanism of MGRAG. Here, the LLM KV cache is stored in a fixed-sized memory, and we replace them with the LRU (Least Recently Used) manner. Based on the cache recompute score  $\phi(\cdot)$ , we will only selectively recompute the  $b\%$  most important tokens' LLM KV matrices, and reuse other tokens' LLM KV matrices that were

stored in Step 2. To be specific, we calculate the cache recompute score  $\phi(t)$  for each token  $t$  regarding three components: (1). Its topological score  $GF_t^q$  regarding its respective entity or relation's topological feature in the query  $q$  specific multimodal KG  $\mathcal{KG}_q$ . (2). Its cross attention score  $\sum_{j=1}^{|q|} a_{tj}$  with tokens of query  $q$ . (3). Its cross attention score  $\sum_{k=1}^{|S_r|} a_{tk}$  with tokens of other entity and relation in  $\mathcal{KG}_q$ .

When processing images, the key events in LLM's image inputs can be fragmented, making the cross-attention score applied by current KV caching mechanisms unable to reflect the importance of image tokens. Hence, we construct a KG  $\mathcal{KG}_q$  to contain the query-related events. As a result, the topological information of  $\mathcal{KG}_q$  can be crucial for reflecting the importance of image tokens regarding certain events. Hence, the first part of our cache recompute score  $\phi(\cdot)$  is the graph's topological feature score  $GF_t^q$  of token  $t$ . As the  $\mathcal{KG}_q$  is constructed with the entities and relations that adaptively extracted from multimodal chunks related to query  $q$ , they are usually small graphs with an average of fewer than 20 entities (see our technical report at <https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf>).

As a result, random walk-based graph scoring algorithms such as PageRank [59] are prone to overfitting and result in suboptimal performance. Consequently, we select the four most commonly used graph statistical features to reflect the topological importance of graph components. Here, we denote  $\mathcal{V}_q$  and  $\mathcal{E}_q$  as the entity and relation set of multimodal KG  $\mathcal{KG}_q$ ,  $\mathcal{N}(\cdot)$  denotes the neighbor node set of a node or a subgraph on  $\mathcal{KG}_q$ :

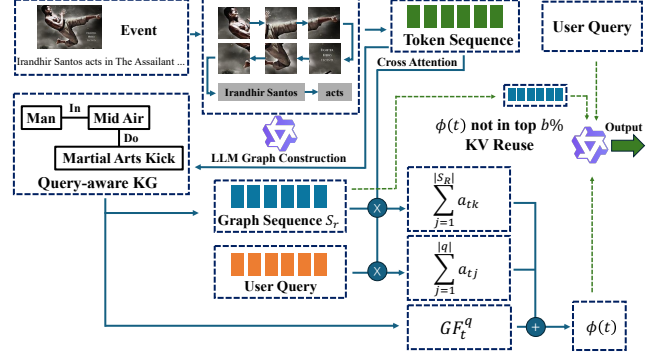
- **Degree Centrality ( $dc$ ):** For  $v \in \mathcal{V}_q$ , degree centrality is  $dc(v) = \frac{\deg(v)}{|\mathcal{V}_q|-1}$  where  $\deg(v)$  is the degree of  $v$  in the multimodal KG  $\mathcal{KG}_q$ . Higher  $dc(v)$  means greater topological importance.
- **Betweenness Centrality ( $bc$ ):** For any  $v \in \mathcal{V}_q$ , we define its betweenness centrality  $bc(v)$  as:

$$bc(v) = \frac{2}{(|\mathcal{V}_q|-1)(|\mathcal{V}_q|-2)} \sum_{x \neq y \neq v, x, y \in \mathcal{V}_q} \frac{|P_{(x,y)}^v|}{|P_{(x,y)}|},$$

Here,  $P_{(x,y)}$  is the set of all paths from node  $x$  to  $y$ , and  $P_{(x,y)}^v \subseteq P_{(x,y)}$  is the subset passing through node  $v$ . This metric measures a node's control over information flow and its role as a bridge in the network; higher values indicate greater importance for graph connectivity.

- **Eigenvector Centrality ( $ec$ ):** Let  $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_n)^\top$  be the principal eigenvector of the adjacency matrix of  $\mathcal{KG}_q$ , and let  $\lambda_v$  denote its component for node  $v$ . The eigenvector centrality is:  $ec(v) = \frac{\lambda_v}{\max(\lambda)}$ . Unlike degree centrality,  $ec(v)$  reflects both direct and indirect influence, recursively rewarding connections to highly central neighbors.
- **Closeness Centrality ( $cc$ ):** For  $v \in \mathcal{V}_q$ ,  $cc(v) = \frac{|\mathcal{V}_q|-1}{\sum_{u \in \mathcal{V}_q, u \neq v} \text{dist}(u,v)}$ , where  $\text{dist}(u,v)$  is the shortest path length between  $u$  and  $v$ . Higher  $cc(v)$  means  $v$  is more centrally located in the graph.

With all these feature scores, we denote their average as the graph topology  $f(u)$  score for node  $u$ :  $f(u) = \frac{dc(u)+bc(u)+ec(u)+cc(u)}{4}$ . After obtaining the graph topology score  $f(\cdot)$  for each graph node, we map each token  $t$  to triplet facts. Here, we denote  $(u, e_{(u,v)}, v)$



**Figure 4: An example of our event-agonistic KV re-computation, where we utilize the graph topological features of the event KG as supplementary knowledge to help mitigate the effects of attention calculation, making the KV Cache Score better reflect the original importance of image tokens.**

as the respective triplet fact that token  $t$  involved in, and map  $t$  to the triplet fact as follow:  $t \in \psi(u) \parallel \psi(e_{(u,v)}) \parallel \psi(v)$ , where  $\psi(\cdot)$  denotes the mapping function from entity or relation to their respective tokens,  $\parallel$  denotes the token concatenation operation, entity nodes  $u, v \in \mathcal{V}_q$ , relation edges  $e_{(u,v)} \in \mathcal{E}_q$ . Then token  $t$ 's graph topological feature  $GF_t^q$  is calculated as the average topological feature score of the two nodes involved in its respective triplet fact:  $GF_t^q = \frac{f(u)+f(v)}{2}$ . Similar to CacheBlend [60], we compute cross-attention scores for our KV cache: specifically, the cross-attention between token  $t$  and the KV cache of query  $q$ , as well as between  $t$  and the KV cache of other tokens.

After obtaining the matched subgraph  $res$ 's graph sequence  $S_{res}$ , we compute the cross attention score  $a$  in a same manner with CacheBlend [60], and obtain the final KV recompute score  $\phi(t)$  of token  $t$  in query  $q$  as:

$$\phi(t) = \sum_{j=1}^{|q|} a_{tj} + \sum_{k=1}^{|S_{res}|} a_{tk} + GF_t^q, \quad (14)$$

where  $|q|$  and  $|S_{res}|$  denotes the number of tokens in query  $q$  and graph sequence  $S_{res}$ , respectively, and  $a_{tj}$  denotes the cross-attention score between token  $t$  and token  $j$ .

### 3.5 Complexity Analysis

Denote the multimodal chunk set as  $|C|$ , the chunk length as  $l_c$ , the embedding dimension as  $d_{emb}$ , the node set of query graph and multimodal KG as  $\mathcal{V}$  and  $\mathcal{V}$ , the number of edges in query graph as  $\mathcal{E}$ , the maximum path context length as  $l_p$ , the query token length as  $l_q$ , the number of chunks used for graph construction as  $k$ , the LLM's hidden states as  $d_{LLM}$ , the number of nodes and edges in multimodal KG as  $\mathcal{V}$  and  $\mathcal{E}$ , the token length of the matched subgraph sequence and query as  $|S_{res}|$  and  $|q|$ . The total offline time complexity of MGRAG is:

$$O(|C|d_{emb}l_c^2),$$

and online time complexity is:

$$O(d_{emb}k l_c^2 + l_p |\mathcal{E}| |\mathcal{V}|^{|\mathcal{V}|+1} + |\mathcal{V}| |\mathcal{E}| + d_{LLM}(|S_{res}|^2 + |q|^2)).$$

Although the time complexity is exponential in the worst case, the average number of nodes and edges in the query graph and

**Table 3: The statistic of MMQA [51], and 7 sub dataset of the MRAMG [61] benchmark. T and I in Query Modality denotes the dataset contains textual and image queries, respectively.**

| Statistic      | MMQA    | Wiki | Wit  | Recipe | Arxiv | Manual | Web   |
|----------------|---------|------|------|--------|-------|--------|-------|
| Query Modality | T / I   | T    | T    | T      | T     | T      | T     |
| Queries        | 951     | 500  | 600  | 2,360  | 200   | 390    | 750   |
| Texts          | 218,285 | 538  | 639  | 1,528  | 101   | 40     | 1,500 |
| Images         | 57,058  | 538  | 639  | 8,569  | 337   | 2,607  | 1,500 |
| Corpus Size    | 22.5M   | 680k | 478k | 1.1M   | 136k  | 420k   | 215k  |
| Query Hops     | 1-2     | 2-5  | 2-4  | 1-6    | 3-4   | 2-6    | 2-3   |
| Avg Query Hops | 1.5     | 3.0  | 2.8  | 3.3    | 3.1   | 3.7    | 2.4   |

multimodal KG is less than 20, so MGRAG’s time cost remains comparable to other baselines. Detailed complexity analysis is available in our technical report at <https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf>.

## 4 EXPERIMENTS

In this section, we outline our experimental setup in Section 4.1, including details on datasets, and evaluation metrics, and report our experimental results in Section 4.2 and Section 4.3 for effectiveness and efficiency, respectively. At last, we conduct the ablation study in Section 4.6 and parameter sensitivity study in Section 4.7.

### 4.1 Experiment Settings

**4.1.1 Datasets.** We select the Arxiv, Manual, Web, Wiki, Wit, and Recipe dataset from the MRAMG benchmark [61], as well as the MMQA dataset [51] to test MGRAG’s performance. The dataset statistics are shown in Table 3, where the MMQA dataset has a large number of source documents, and its queries can consist of image and text pairs, making it a challenging dataset for evaluating RAG models. The MRAMG benchmark is the most up-to-date benchmark of the multimodal QA task, and its six datasets are built upon source data from various domains, compared to the MMQA dataset, it has a much smaller number of source documents, and only consists of textual queries.

**4.1.2 Evaluation Metrics.** We employ four metrics to evaluate the performance of MGRAG and baselines via the RAGAS evaluation framework [16], as follows:

- (1) **Semantic Similarity (SIM.):** Measures the vector’s cosine similarity between the generated answer and the gold answer, we apply the bge-base-en-v1.5 [58] for vector encoding.
- (2) **Accuracy (ACC.):** Measures the proportion of correct answers among all answers generated by a model.
- (3) **ROUGE-L (ROG.)** [40]: Evaluates the quality of generated text by comparing the longest common subsequence (LCS) between the generated answer  $G$  and the ground-truth answer  $Y$ .
- (4) **BLEU-1 (BLU.)** [45]: Measures the quality of the generated answer  $G$  by computing the modified unigram precision with respect to the ground-truth answer  $Y$ .

**4.1.3 Baselines.** As shown in Table 4 and Table 5, we compare MGRAG against a broad set of retrieval, subgraph matching, KG construction, and KV cache baselines.

- **RAG Systems:** We compare MGRAG’s performance with 5 RAG systems in Table 4. Among these systems, VanillaRAG [20] is a similarity-based RAG system that retrieves top- $k$  semantic-nearest chunks with a dense encoder and directly feeds them to the LLM. R1-Searcher [49] fine-tunes the retrieval and reasoning pipeline of VanillaRAG with reinforcement learning to improve multi-step question answering. GraphRAG [15] constructs a global knowledge graph from the corpus, performs community detection, and generates community summaries as graph-level contexts for QA. LightRAG [23] simplifies GraphRAG by skipping community construction and instead retrieving high-scoring nodes and their neighborhoods to reduce retrieval cost. HippoRAG2 [25] further improves graph-based retrieval by indexing full passages as nodes and using Personalized PageRank to select a fixed number of text or image chunks.
- **KG Construction methods:** We compare MGRAG’s performance with 2 KG construction methods in Table 5. Stanford OpenIE (O.IE) [6] extracts entities and relations using pattern-based information extraction without LLMs, while STAT [57] builds KGs via term-frequency based statistical entity extraction.
- **Subgraph Matching methods:** We compare MGRAG’s performance with 9 subgraph matching methods in Table 5. Among these systems, VF2 [13] is a classic exact subgraph-isomorphism solver based on depth-first backtracking with strong feasibility pruning, and ISMAGS [28] extends VF2 with symmetry-aware search to avoid redundant isomorphic embeddings. SM [35] relaxes discrete matching to a spectral optimization over an affinity matrix, VJ [17] initializes from seeded correspondences and iteratively expands them, RRWM [11] performs reweighted random walks on the association graph to obtain robust soft correspondences, and IPFP [36] solves a quadratic assignment relaxation via an integer-projected fixed-point iteration. BGE [58] measures graph similarity in a learned embedding space by encoding entities and relations into dense vectors, SLOAlign [52] jointly refines graph structure and computes an optimal-transport based alignment between attributed graphs, and FGWAlign [53] uses a fused Gromov-Wasserstein distance to couple node attributes and topology for graph edit distance style alignment.
- **KV Caching mechanisms:** We compare MGRAG’s performance with 2 KV Cache mechanisms in Table 7. vLLM [34] provides prefix-based paged-attention KV caching to accelerate decoding, while CacheBlend [60] is a position-independent method that scores tokens by cross-attention importance and recomputes only the most salient ones without exploiting graph structure.

For VanillaRAG, we implement it with the langchain framework and FAISS vector database [14]. For GraphRAG, we use the implementation from [1] which optimizes its original code and achieves better time efficiency while not affect the performance; For all other baselines, we use their official implementations. For all similarity-based retrieval scenarios, we encode all images’ captions to represent them for similarity based retrieval as in [22], for achieving better performance. We deploy the Qwen-2.5-VL-7B-Instruct backbone with vLLM [34], and use vLLM [34] to accelerate all tested baselines other than MGRAG.

**4.1.4 Hyperparameter and Hardware Settings.** We set the retrieval top- $k$  value for all RAG models to 5, aligning with the

**Table 4: The experiment result on MMQA [51], and 7 sub dataset of the MRAMG [61] benchmark, all experiments were carried out with Qwen2.5-VL-7B-Instruct backbone. "-" denotes the experiment cost longer than 3 days.**

| Dataset | Metric | Similarity-based |        | Graph-based |        |        |               |
|---------|--------|------------------|--------|-------------|--------|--------|---------------|
|         |        | Vanilla.         | R1.    | Graph.      | Light. | Hippo. | MGRAG         |
| MMQA    | SIM.   | 0.8362           | 0.5478 | -           | -      | 0.8381 | <b>0.8533</b> |
|         | ACC.   | 0.7053           | 0.3567 | -           | -      | 0.6335 | <b>0.7442</b> |
|         | ROG.   | 0.5942           | 0.1041 | -           | -      | 0.6659 | <b>0.6763</b> |
|         | BLEU   | 0.6645           | 0.0817 | -           | -      | 0.5917 | <b>0.7215</b> |
| Arxiv   | SIM.   | 0.8664           | 0.7051 | 0.7943      | 0.7404 | 0.8293 | <b>0.8760</b> |
|         | ACC.   | 0.6638           | 0.3775 | 0.2350      | 0.3813 | 0.4688 | <b>0.6800</b> |
|         | ROG.   | 0.2713           | 0.1189 | 0.1553      | 0.1580 | 0.2331 | <b>0.2726</b> |
|         | BLEU   | 0.2452           | 0.1092 | 0.1690      | 0.1370 | 0.1281 | <b>0.2498</b> |
| Manual  | SIM.   | 0.8166           | 0.6046 | 0.5453      | 0.7255 | 0.6852 | <b>0.8221</b> |
|         | ACC.   | 0.5881           | 0.2071 | 0.1436      | 0.4096 | 0.2641 | <b>0.6173</b> |
|         | ROG.   | 0.3509           | 0.1078 | 0.0974      | 0.1595 | 0.2047 | <b>0.3839</b> |
|         | BLEU   | 0.2767           | 0.0740 | 0.0718      | 0.1385 | 0.1314 | <b>0.3093</b> |
| Web     | SIM.   | 0.6937           | 0.6620 | 0.5470      | 0.5513 | 0.5642 | <b>0.7009</b> |
|         | ACC.   | 0.4703           | 0.4200 | 0.1053      | 0.2000 | 0.3607 | <b>0.5123</b> |
|         | ROG.   | 0.1508           | 0.1142 | 0.1133      | 0.0905 | 0.0616 | <b>0.1664</b> |
|         | BLEU   | 0.0326           | 0.0785 | 0.1031      | 0.0206 | 0.0133 | <b>0.1456</b> |
| Wiki    | SIM.   | 0.8824           | 0.5478 | 0.6966      | 0.7517 | 0.8204 | <b>0.8838</b> |
|         | ACC.   | 0.7800           | 0.3567 | 0.2000      | 0.3660 | 0.6500 | <b>0.7845</b> |
|         | ROG.   | 0.3559           | 0.1418 | 0.1361      | 0.2008 | 0.3042 | <b>0.3579</b> |
|         | BLEU   | 0.3053           | 0.1146 | 0.1007      | 0.1863 | 0.1851 | <b>0.3174</b> |
| Wit     | SIM.   | 0.7851           | 0.6110 | 0.6211      | 0.6973 | 0.7230 | <b>0.8033</b> |
|         | ACC.   | 0.6675           | 0.6017 | 0.1500      | 0.2483 | 0.5138 | <b>0.6967</b> |
|         | ROG.   | 0.3255           | 0.1307 | 0.1319      | 0.2416 | 0.2712 | <b>0.3589</b> |
|         | BLEU   | 0.2095           | 0.0934 | 0.1142      | 0.1928 | 0.1200 | <b>0.2382</b> |
| Recipe  | SIM.   | 0.8599           | 0.6155 | 0.8166      | 0.8118 | 0.7792 | <b>0.8673</b> |
|         | ACC.   | 0.6525           | 0.2180 | 0.2771      | 0.3334 | 0.4917 | <b>0.6814</b> |
|         | ROG.   | 0.3714           | 0.1065 | 0.1745      | 0.2128 | 0.3303 | <b>0.4220</b> |
|         | BLEU   | 0.2599           | 0.0773 | 0.1527      | 0.1648 | 0.1941 | <b>0.3102</b> |

settings of HippoRAG2 [25]. We apply the hybrid search and local model for LightRAG [23] and GraphRAG [15], respectively, as they are the most complete search mode and achieve best performance. We apply LightRAG’s local search mode for the text classification task, as it achieves the best effectiveness on the two datasets tested. For HippoRAG2 [25], we set its hyperparameters to default, aligning with its original paper and code. For our MGRAG, the text chunks’ length is set to 256, and token overlap between chunks to 32, to align with the setting of HippoRAG2. All our experiments were carried out and evaluated with Qwen2.5-VL-7B-Instruct [7] LLM backbone, as Qwen is the best open-sourced multimodal LLM model according to huggingface LLM-leaderboard [29]. We apply the Contriever [30] embedding model for HippoRAG2’s dense passage retrieval procedure, and bge-base-en-v1.5 [58] embedding model for other baselines’ LLM embedding, align with HippoRAG2’s [25] setting. All experiments are conducted on an Intel(R) Xeon(R) Gold 5220R @ 2.20GHz CPU and a single NVIDIA A100-SXM4-40GB GPU. Other hyperparameters of models were set as default by their codes.

## 4.2 Effectiveness Evaluation

As shown in Table 4 and Table 7, MGRAG achieves the overall best performance over all 7 datasets, surpasses all tested baselines. Based on the experiment results, we can have the following observations:

**Table 5: Ablation experiments of MGRAG with 2 multimodal KG construction and 5 subgraph matching baselines, all experiments were carried out with Qwen2.5-VL-7B-Instruct LLM backbone.**

| Dataset | Metric | MMKGC   |       | Subgraph Matching |                   |       |       |              |                |              |              | Ours         |              |
|---------|--------|---------|-------|-------------------|-------------------|-------|-------|--------------|----------------|--------------|--------------|--------------|--------------|
|         |        | Non LLM |       | Exact Match       | Approximate Match |       |       |              | Semantic Match |              |              |              |              |
|         |        | OLE     | STAT  | VF2               | ISMAGS            | SM    | VJ    | RRWM         | IPFP           | BGE          | SLOT         |              | FGW          |
| MMQA    | SIM.   | 0.642   | 0.655 | <u>0.851</u>      | 0.833             | 0.832 | 0.812 | 0.838        | 0.773          | 0.832        | 0.838        | 0.837        | <b>0.853</b> |
|         | ACC.   | 0.571   | 0.602 | <u>0.735</u>      | 0.706             | 0.702 | 0.715 | 0.698        | 0.633          | 0.703        | 0.700        | 0.697        | <b>0.744</b> |
|         | ROG.   | 0.510   | 0.519 | <u>0.673</u>      | 0.638             | 0.632 | 0.646 | 0.647        | 0.601          | 0.635        | 0.650        | 0.647        | <b>0.676</b> |
|         | BLEU   | 0.483   | 0.500 | <u>0.620</u>      | 0.590             | 0.582 | 0.606 | 0.599        | 0.585          | 0.586        | 0.602        | 0.598        | <b>0.722</b> |
| Arxiv   | SIM.   | 0.618   | 0.640 | <u>0.871</u>      | 0.868             | 0.866 | 0.872 | <u>0.874</u> | 0.814          | <u>0.874</u> | 0.872        | 0.868        | <b>0.876</b> |
|         | ACC.   | 0.526   | 0.576 | <u>0.650</u>      | 0.643             | 0.670 | 0.667 | <u>0.675</u> | 0.581          | 0.668        | 0.653        | 0.651        | <b>0.680</b> |
|         | ROG.   | 0.177   | 0.184 | <u>0.270</u>      | 0.266             | 0.263 | 0.269 | 0.257        | 0.212          | <u>0.270</u> | 0.269        | 0.264        | <b>0.273</b> |
|         | BLEU   | 0.138   | 0.145 | <u>0.239</u>      | 0.231             | 0.244 | 0.245 | 0.243        | 0.203          | <u>0.244</u> | 0.233        | 0.230        | <b>0.250</b> |
| Manual  | SIM.   | 0.520   | 0.509 | <u>0.817</u>      | 0.817             | 0.813 | 0.808 | 0.811        | 0.800          | 0.815        | 0.813        | <u>0.820</u> | <b>0.822</b> |
|         | ACC.   | 0.459   | 0.443 | <u>0.612</u>      | 0.612             | 0.614 | 0.608 | 0.614        | 0.594          | <u>0.615</u> | 0.611        | 0.600        | <b>0.617</b> |
|         | ROG.   | 0.245   | 0.234 | <u>0.377</u>      | 0.382             | 0.378 | 0.372 | 0.377        | 0.369          | 0.375        | 0.375        | 0.374        | <b>0.384</b> |
|         | BLEU   | 0.209   | 0.182 | <u>0.306</u>      | <u>0.307</u>      | 0.303 | 0.298 | 0.303        | 0.289          | 0.303        | 0.301        | 0.292        | <b>0.309</b> |
| Web     | SIM.   | 0.466   | 0.485 | <u>0.677</u>      | 0.675             | 0.676 | 0.679 | 0.675        | 0.666          | <u>0.687</u> | 0.676        | 0.676        | <b>0.701</b> |
|         | ACC.   | 0.290   | 0.298 | <u>0.507</u>      | 0.509             | 0.507 | 0.508 | 0.503        | 0.493          | <u>0.510</u> | 0.498        | 0.480        | <b>0.512</b> |
|         | ROG.   | 0.052   | 0.101 | <u>0.156</u>      | 0.155             | 0.156 | 0.160 | <u>0.157</u> | 0.149          | 0.156        | 0.154        | 0.147        | <b>0.166</b> |
|         | BLEU   | 0.020   | 0.095 | <u>0.088</u>      | 0.079             | 0.056 | 0.056 | 0.066        | 0.055          | <u>0.095</u> | 0.116        | 0.096        | <b>0.146</b> |
| Wiki    | SIM.   | 0.504   | 0.525 | <u>0.877</u>      | 0.879             | 0.878 | 0.873 | 0.878        | 0.869          | 0.878        | 0.879        | 0.878        | <b>0.884</b> |
|         | ACC.   | 0.498   | 0.510 | <u>0.752</u>      | <u>0.754</u>      | 0.750 | 0.751 | 0.743        | 0.735          | 0.746        | 0.749        | 0.747        | <b>0.785</b> |
|         | ROG.   | 0.230   | 0.253 | <u>0.352</u>      | 0.355             | 0.353 | 0.349 | 0.355        | 0.348          | <u>0.355</u> | <u>0.356</u> | 0.352        | <b>0.358</b> |
|         | BLEU   | 0.200   | 0.218 | <u>0.300</u>      | <u>0.303</u>      | 0.298 | 0.297 | 0.301        | 0.296          | 0.301        | 0.301        | 0.300        | <b>0.317</b> |
| Wit     | SIM.   | 0.492   | 0.489 | <u>0.799</u>      | 0.797             | 0.784 | 0.786 | 0.795        | 0.792          | 0.789        | 0.793        | 0.794        | <b>0.803</b> |
|         | ACC.   | 0.465   | 0.453 | <u>0.693</u>      | <u>0.694</u>      | 0.686 | 0.691 | 0.679        | 0.678          | 0.689        | 0.693        | 0.691        | <b>0.697</b> |
|         | ROG.   | 0.233   | 0.205 | <u>0.354</u>      | 0.355             | 0.353 | 0.356 | 0.347        | 0.346          | <u>0.356</u> | 0.351        | 0.349        | <b>0.359</b> |
|         | BLEU   | 0.210   | 0.198 | <u>0.234</u>      | 0.231             | 0.229 | 0.227 | 0.230        | 0.231          | 0.230        | 0.227        | 0.223        | <b>0.238</b> |
| Recipe  | SIM.   | 0.551   | 0.545 | <u>0.859</u>      | 0.861             | 0.862 | 0.857 | 0.852        | 0.847          | <u>0.864</u> | 0.861        | 0.860        | <b>0.867</b> |
|         | ACC.   | 0.425   | 0.401 | <u>0.679</u>      | 0.668             | 0.678 | 0.672 | 0.670        | 0.661          | <u>0.674</u> | 0.675        | 0.677        | <b>0.681</b> |
|         | ROG.   | 0.211   | 0.198 | <u>0.413</u>      | 0.407             | 0.411 | 0.413 | 0.406        | 0.395          | <u>0.415</u> | 0.412        | 0.409        | <b>0.422</b> |
|         | BLEU   | 0.202   | 0.172 | <u>0.304</u>      | 0.302             | 0.305 | 0.295 | 0.298        | 0.221          | <u>0.308</u> | 0.306        | 0.305        | <b>0.310</b> |

Firstly, compared to the previous state-of-the-art systems, MGRAG achieves a larger effectiveness improvement (5.5% on ACC.) on the large MMQA dataset than on other datasets. This is because, large external multimodal sources means a large amount of noise for each retrieval, and an accurate and query-aware graph construction is crucially important in this scenario.

Secondly, although lacking the ability of fine-grained retrieval, the Similarity-based RAG systems can still achieve comparable or better performance on certain tested datasets. This is because the Graph-based RAG baselines’ graph construction steps are query-agnostic, which limits their ability to utilize the fine-grained multimodal source information, leading to high information loss. Consequently, as they lose the fine-grained multimodal information early in the graph construction step, their later graph retrieval step is also affected, leading to suboptimal performance. For our MGRAG, the graphs constructed are smaller but query-aware and more tailored for the respective query, enabling the system to better utilize the fine-grained multimodal source information.

Lastly, according to the experiment results on different hop queries of MRAMG benchmark in Table 9, MGRAG achieves better performance on long-hop queries. This demonstrates that MGRAG’s subgraph matching mechanism can be redundant and costly when the query only requires simple retrieval, and crucial for handling complex queries.

## 4.3 Efficiency Evaluation

We evaluate the efficiency of MGRAG in comparison to both similarity-based and graph-based RAG baselines, using the challenging MMQA dataset. Results are summarized in Table 6 and Table 7. Table 6

**Table 6: System cost on MMQA dataset, where "-" denotes the experiment cost longer than 3 days, and AVG Token Cost denotes the average token cost per query.**

| Category         | Model       | Time Cost | AVG Token Cost |
|------------------|-------------|-----------|----------------|
| Similarity-based | VanillaRAG  | 2.6 h     | 1132.1         |
|                  | R1-Searcher | 2.9 h     | 1835.4         |
| Graph-based      | GraphRAG    | -         | -              |
|                  | LightRAG    | -         | -              |
|                  | HippoRAG2   | 7.5 h     | 292,436.2      |
|                  | MGRAG       | 4.3 h     | 4613.4         |

**Table 7: Efficiency and effectiveness comparison between MGRAG and other KV cache mechanisms on MMQA dataset. Where V. and H. denote these systems cope with similarity-based VanillaRAG and graph-based HippoRAG2 framework.**

| System          | SIM.   | ACC.   | ROG.   | BLEU   | TTFT  | Time Cost |
|-----------------|--------|--------|--------|--------|-------|-----------|
| vLLM (V.)       | 0.8315 | 0.6593 | 0.6277 | 0.5867 | 3.2 s | 4.5 h     |
| CacheBlend (V.) | 0.8315 | 0.6601 | 0.6417 | 0.5964 | 3.1 s | 4.5 h     |
| vLLM (H.)       | 0.8381 | 0.6335 | 0.6659 | 0.5917 | 4.6 s | 8.8 h     |
| CacheBlend (H.) | 0.8415 | 0.6267 | 0.6636 | 0.5917 | 4.4 s | 8.3 h     |
| MGRAG           | 0.8533 | 0.7442 | 0.6763 | 0.7215 | 3.2 s | 4.3 h     |

**Table 8: MGRAG’s breakdown time cost for each module. Where KGC denotes multimodal KG construction time and token cost, Match denotes subgraph matching time cost, QA denotes LLM QA time and token cost, and E2E denotes end-to-end time and token cost.**

| Dataset | # Query | KGC           | Match | QA            | E2E            |
|---------|---------|---------------|-------|---------------|----------------|
| MMQA    | 951     | 193.0 / 3,085 | 0.14  | 8.9 / 1,431   | 248.1 / 4,516  |
| Arxiv   | 200     | 61.9 / 3,946  | 0.23  | 16.2 / 2,944  | 83.9 / 6,890   |
| Manual  | 390     | 76.1 / 4,285  | 0.04  | 23.9 / 2,556  | 109.7 / 6,841  |
| Web     | 750     | 143.8 / 4,623 | 0.27  | 42.9 / 3,101  | 196.5 / 7,723  |
| Wiki    | 500     | 164.2 / 6,629 | 0.30  | 52.1 / 6,267  | 241.5 / 12,896 |
| Wit     | 600     | 140.6 / 7,660 | 0.21  | 42.6 / 6,539  | 196.5 / 14,199 |
| Recipe  | 2,360   | 525.7 / 4,351 | 1.74  | 191.3 / 3,173 | 808.7 / 7,525  |

presents the overall system cost in terms of processing time and average token cost per query. MGRAG achieves a total completion time of 4.3 hours and an average token cost of 4,613.4 per query, which is over 60 times lower than HippoRAG2 (292,436.2 tokens/query) and significantly more efficient in terms of wall-clock time (4.3 h vs. 7.5 h). Notably, other graph-based systems such as GraphRAG and LightRAG are unable to complete the MMQA experiment within three days, highlighting their scalability limitations. While similarity-based baselines (VanillaRAG, R1-Searcher) process the dataset faster and with slightly lower token costs (our additional 3500 more tokens only cost 0.035 USD for GPT4o), they lack the fine-grained and entity-aware retrieval capabilities of MGRAG, which are critical for complex multimodal QA tasks and achieve lower performance shown in Table 4.

We also add fine-grained breakdown time and token cost statistics of MGRAG’s each parts in Table 8. According to the results, the repeated LLM calls for entity and relation extraction is the key efficiency bottleneck of our MGRAG model, and also a future research direction for improving the efficiency of graph-based RAG models. But in return, by adaptively building compact, relevant knowledge graphs using only the most pertinent multimodal chunks, MGRAG

**Table 9: Performance comparison between VanillaRAG and MGRAG on different hop queries from the MRAMG benchmark. Times are in minutes.**

| # Hop | # Q   | VanillaRAG |       |       |       | MGRAG |       |       |       |
|-------|-------|------------|-------|-------|-------|-------|-------|-------|-------|
|       |       | Time       | ACC.  | ROG.  | BLEU  | Time  | ACC.  | ROG.  | BLEU  |
| 1-hop | 201   | 0.13       | 0.613 | 0.379 | 0.240 | 0.35  | 0.624 | 0.425 | 0.286 |
| 2-hop | 1,237 | 0.10       | 0.588 | 0.304 | 0.193 | 0.37  | 0.601 | 0.335 | 0.230 |
| 3-hop | 2,357 | 0.16       | 0.633 | 0.329 | 0.238 | 0.41  | 0.659 | 0.355 | 0.260 |
| 4-hop | 499   | 0.19       | 0.697 | 0.338 | 0.259 | 0.44  | 0.712 | 0.373 | 0.293 |
| 5-hop | 286   | 0.18       | 0.678 | 0.319 | 0.205 | 0.45  | 0.733 | 0.392 | 0.282 |
| 6-hop | 220   | 0.18       | 0.661 | 0.305 | 0.176 | 0.43  | 0.738 | 0.377 | 0.249 |

**Table 10: Ablation experiment of MGRAG on MMQA.**

| Model                | SIM.   | ACC.   | ROG.   | BLEU   | Time Cost |
|----------------------|--------|--------|--------|--------|-----------|
| MGRAG w.o. GM        | 0.8362 | 0.7260 | 0.6657 | 0.5774 | 4.4 h     |
| MGRAG w. MM Encoder  | 0.8513 | 0.7376 | 0.6744 | 0.7008 | 4.5 h     |
| Vanilla MGRAG w. k=5 | 0.8260 | 0.6887 | 0.6266 | 0.5773 | 4.7 h     |
| MGRAG                | 0.8533 | 0.7442 | 0.6763 | 0.7215 | 4.3 h     |

avoids the costly and redundant graph expansion that plagues other graph-based approaches and achieves better effectiveness.

#### 4.4 Subgraph Matching Experiments

To compare MGRAG’s path-based semantic subgraph matching method with other applicable subgraph matching methods, we conduct extensive experiments on MMQA dataset in Table 5. As the query graph and data graph are all query specific in our subgraph matching scheme, any training-based method is not applicable, hence we only select exact matching algorithms VF2 [13], and IS-MAGS [28], the approximate matching algorithms SM [35], VJ [17], RRWM [11], and IPFP [36], and the semantic matching algorithms BGE [58], SLOAlign [52], FGWAlign [53] for comparison. Based on the results in Table 5, we can have the following observations:

Firstly, exact matching methods achieve suboptimal performance, demonstrating that exact topological matching ignores the semantic correlation between the query and the KG, which leads to inaccurate graph matching. Secondly, on most tested datasets, semantic matching methods achieve the second-best performance, which demonstrates the importance of semantic correlation between the query and the KG for accurate graph matching. However, they also achieve worse efficiency than MGRAG, as they ignore the path-based feature during node matching, which is crucial for accurate semantic subgraph matching in our scenario.

#### 4.5 KV Cache Experiments

To systematically assess the efficiency and effectiveness of the proposed graph-aware KV cache mechanism, we compare the mechanism against two representative KV cache baselines, vLLM and CacheBlend, each equipped with both similarity-based (VanillaRAG) and graph-based (HippoRAG2) retrieval modules. The results are summarized in Table 7, where all experiments are conducted on the MMQA dataset under the same hardware and model backbone (Qwen2.5-VL-7B-Instruct), ensuring fair and consistent evaluation. Table 7 summarizes the results across metrics of semantic similarity (SIM.), accuracy (ACC.), ROUGE-L (ROG.), BLEU, time-to-first-token (TTFT), and total time cost.

**Table 11: The inverse ablation of MGRAG’s path-based semantic subgraph matching (PSGM) and graph-based KV (GKV) cache mechanism, with two best performing RAG system HippoRAG2 [25] and VanillaRAG [20] on MMQA.**

| Model             | SIM.          | ACC.          | ROG.          | BLEU          | Time         |
|-------------------|---------------|---------------|---------------|---------------|--------------|
| Hippo.            | 0.8381        | 0.6335        | 0.6659        | 0.5917        | 7.5 h        |
| Hippo.+GKV        | 0.8392        | 0.6855        | 0.6651        | 0.7013        | 7.6 h        |
| Hippo.+PSGM       | 0.8416        | <u>0.7372</u> | <u>0.6720</u> | <u>0.7128</u> | <u>4.1 h</u> |
| Hippo.+PSGM+GKV   | <b>0.8533</b> | <b>0.7442</b> | <b>0.6763</b> | <b>0.7215</b> | 4.3 h        |
| Vanilla.          | 0.8362        | 0.7053        | 0.5942        | 0.6645        | <b>2.6 h</b> |
| Vanilla.+PSGM     | 0.8460        | 0.7175        | 0.6466        | 0.6973        | 4.7 h        |
| Vanilla.+PSGM+GKV | 0.8485        | 0.7232        | 0.6563        | 0.7001        | 4.8 h        |
| <b>MGRAG</b>      | <b>0.8533</b> | <b>0.7442</b> | <b>0.6763</b> | <b>0.7215</b> | 4.3 h        |

According to the results in Table 7, MGRAG consistently achieves superior efficiency and answer quality compared to all vLLM and CacheBlend variants. While vLLM and CacheBlend are fast when sequepped with both similarity-based VanillaRAG, they incur significant overhead and reduced answer quality when applied to graph-based retrieval. In contrast, MGRAG’s event-aware, graph-structure-guided KV caching enables both fast inference and high answer quality across all metrics and retrieval paradigms.

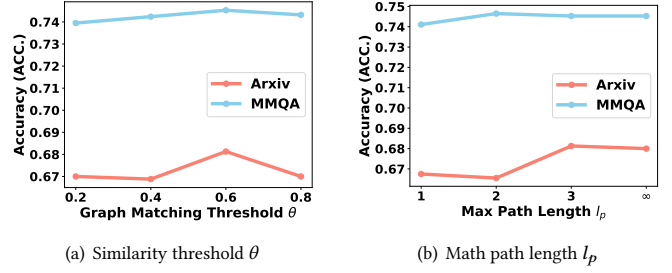
These results highlight the advantage of our approach: by leveraging the structural and semantic properties of the constructed knowledge graph, MGRAG’s KV cache mechanism delivers state-of-the-art performance.

## 4.6 Ablation Study

**4.6.1 Model ablation.** To analyze MGRAG’s performance, we conduct an extensive ablation study on the MMQA dataset, the ablation models tested are listed as follows:

- **MGRAG w.o. GM:** We remove MGRAG’s semantic path-based subgraph matching algorithm, and construct query-specific graph with a fixed number (the maximum number of coarse retrieval) of multimodal chunks.
- **MGRAG w. MM Encoder:** We directly use CLIP [46] to encode multimodal contents, not using image captions as a proxy for multimodal input in coarse-grained ranking.
- **Vanilla MGRAG:** We directly use our path-based semantic subgraph matching method and the graph-based KV cache mechanism with the VanillaRAG pipeline, and set the retrieval amount  $k = 5$  (same as our MGRAG’s coarse-grained retrieval amount).

Based on the results in Table 10, we can have the following observations regarding MGRAG: Firstly, **Vanilla MGRAG** achieves the worst effectiveness, which demonstrates that our MGRAG’s coarse-to-fine-grained adaptive retrieval pipeline can reduce the retrieval noise by avoiding a large amount of noisy retrieval. Secondly, **MGRAG w.o. GM** achieves the second worst effectiveness, which demonstrates that using a fixed number of multimodal chunks to construct KG can introduce noise or insufficient knowledge, and consequently lead to suboptimal effectiveness. Lastly, aligned with the finding in [22], **MGRAG w. MM Encoder** achieves worse performance than MGRAG, which uses image caption as a proxy for multimodal input. This demonstrates the limited encoding ability of current multimodal encoders, which causes more information



**Figure 5: Parameter sensitivity of  $\theta$  and  $l_p$ .**

loss than converting the image to text and using text encoders to generate representation vectors.

**4.6.2 Inverse Ablations.** We carry out inverse ablations of MGRAG’s three stages on the MMQA (the easiest) dataset in Table 11, where we integrate our path-based semantic subgraph matching (Namely PSGM) and graph-based KV cache (Namely GKV) mechanism with the previously best-performed graph-based RAG model HippoRAG2 [25], and non graph-based model VanillaRAG [20]. According to the results, each of our proposed path-based semantic subgraph matching (PSGM) and graph-based KV cache mechanism (GKV) can further improve the performance of these two tested baselines, and achieve the best performance when combined, demonstrating the effectiveness and generalizability of our proposed mechanisms. Moreover, it is also worth noting that the performance of HippoRAG2+PSGM+GKV is the same as our MGRAG, since our MGRAG applied the same KG construction method as HippoRAG2, adding the PSGM and GKV mechanisms to HippoRAG2 makes the setting identical to our MGRAG.

## 4.7 Parameter Sensitivity

In this subsection, we analyze the parameter sensitivity of MGRAG’s unique hyperparameter, the graph matching similarity threshold  $0 < \theta < 1$ , and the max semantic path length  $l_p$ , for which we test values from  $\{1, 2, 3, \infty\}$ . For MGRAG’s other parameters, we align them with our tested baselines. According to Figure 5, we can have the following observations: Firstly, for the graph similarity threshold  $\theta$ , when the threshold is 0.6, MGRAG tends to achieve better performance. This demonstrates that a moderate similarity threshold can help MGRAG to better balance the precision and recall of the subgraph matching, which is crucial for accurate query-graph mapping and consequently better retrieval performance. Secondly, for the maximum path length  $l_p$ , when  $l_p$  is set to 3, MGRAG achieves the best performance, which demonstrates that considering paths with a maximum length of 3 can effectively capture the relevant semantic relationships in the knowledge graph without introducing excessive noise or computational overhead.

## 5 CONCLUSION

We present MGRAG, a graph-based multimodal RAG framework that tackles limitations in query awareness, flexible graph matching, and KV cache management. By integrating query-driven KG construction, path-based semantic subgraph matching, and graph-aware KV caching, MGRAG achieves higher accuracy and efficiency than existing baselines, especially on complex multi-hop queries.

Our results demonstrate the value of principled graph methods for retrieval-augmented generation with large language models.

## REFERENCES

- [1] [n.d.]. GitHub - gusye1234/nano-graphrag: A simple, easy-to-hack GraphRAG implementation — github.com. <https://github.com/gusye1234/nano-graphrag>. [Accessed 17-10-2025].
- [2] Mohammad Mahdi Abootorabi, Amirhosein Zobeiri, Mahdi Dehghani, Mohammadali Mohammadkhani, Bardia Mohammadi, Omid Ghahroodi, Mahdiah Soleymani Baghshah, and Ehsaneddin Asgari. 2025. Ask in any modality: A comprehensive survey on multimodal retrieval-augmented generation. *arXiv preprint arXiv:2502.08826* (2025).
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [4] Omar Adjali, Olivier Ferret, Sahar Ghannay, and Hervé Le Borgne. 2024. Multi-level information retrieval augmented generation for knowledge-based visual question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 16499–16513.
- [5] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [6] Gabor Angeli, Melvin Jose Johnson Premkumar, and Christopher D Manning. 2015. Leveraging linguistic structure for open domain information extraction. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 344–354.
- [7] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).
- [8] Yukun Cao, Zengyi Gao, Zhiyang Li, Xike Xie, and S Kevin Zhou. 2025. LEGO-GraphRAG: Modularizing Graph-based Retrieval-Augmented Generation for Design Space Exploration. *Proceedings of the VLDB Endowment* 18 (2025).
- [9] Thierry Champion, Luigi De Pascale, and Petri Juutinen. 2008. The inf-Wasserstein Distance: Local Solutions and Existence of Optimal Transport Maps. *SIAM Journal on Mathematical Analysis* 40, 1 (2008), 1–20.
- [10] Yihua Cheng, Yuhua Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).
- [11] Minsu Cho, Jungmin Lee, and Kyoung Mu Lee. 2010. Reweighted random walks for graph matching. In *European conference on Computer vision*. Springer, 492–505.
- [12] circlemind ai. 2025. *fast-graphrag*. <https://github.com/circlemind-ai/fast-graphrag>
- [13] Luigi P Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. A (sub) graph isomorphism algorithm for matching large graphs. *IEEE transactions on pattern analysis and machine intelligence* 26, 10 (2004), 1367–1372.
- [14] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). *arXiv:2401.08281 [cs.LG]*
- [15] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [16] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 150–158.
- [17] Stefan Fankhauser, Kaspar Riesen, and Horst Bunke. 2011. Speeding up graph edit distance computation through fast bipartite matching. In *International Workshop on Graph-Based Representations in Pattern Recognition*. Springer, 102–111.
- [18] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature* 630, 8017 (2024), 625–630.
- [19] Sensen Gao, Shanshan Zhao, Xu Jiang, Lunhao Duan, Yong Xien Chng, Qing-Guo Chen, Weihua Luo, Kaifu Zhang, Jia-Wang Bian, and Mingming Gong. 2025. Scaling Beyond Context: A Survey of Multimodal Retrieval-Augmented Generation for Document Understanding. *arXiv:2510.15253 [cs.CL]* <https://arxiv.org/abs/2510.15253>
- [20] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).
- [21] M.R. Garey and D.S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman.
- [22] Ankush Gola. 2024. Multi-modal rag on Slide Decks. <https://blog.langchain.com/multi-modal-rag-template/>
- [23] Zirui Guo, Lianhao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. In *Proceedings of the 2025 conference on empirical methods in natural language processing*.
- [24] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *arXiv preprint arXiv:2405.14831* (2024).
- [25] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. *Forty-second International Conference on Machine Learning* (2025).
- [26] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024).
- [27] Darryl Hannan, Akshay Jain, and Mohit Bansal. 2020. Mnymodalqa: Modality disambiguation and qa over diverse inputs. In *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 7879–7886.
- [28] Maarten Houbraken, Sofie Demeyer, Tom Michoel, Pieter Audenaert, Didier Colle, and Mario Pickavet. 2014. The Index-based Subgraph Matching Algorithm with General Symmetries (ISMAGS): exploiting symmetry for faster subgraph enumeration. *PloS one* 9, 5 (2014), e97896.
- [29] huggingface. 2025. *llm-leaderboard*. [https://huggingface.co/spaces/open-llm-leaderboard/open\\_llm\\_leaderboard/#/](https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard/#/)
- [30] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. [n.d.]. Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research* ([n. d.]).
- [31] Xun Jian, Zhiyuan Li, and Lei Chen. 2023. Suff: Accelerating subgraph matching with historical data. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1699–1711.
- [32] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 1658–1677. <https://aclanthology.org/2024.acl-long.91>
- [33] Jinhao Jiang, Kun Zhou, Xin Zhao, and Ji-Rong Wen. 2023. UniKGQA: Unified Retrieval and Reasoning for Solving Multi-hop Question Answering Over Knowledge Graph. In *The Eleventh International Conference on Learning Representations*.
- [34] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [35] Marius Lordeanu and Martial Hebert. 2005. A spectral technique for correspondence problems using pairwise constraints. In *Tenth IEEE International Conference on Computer Vision (ICCV’05) Volume 1, Vol. 2*. IEEE, 1482–1489.
- [36] Marius Lordeanu, Martial Hebert, and Rahul Sukthankar. 2009. An integer projected fixed point method for graph matching and map inference. *Advances in neural information processing systems* 22 (2009).
- [37] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [38] Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanhao Xu, Xuejia Chen, Nicole HU, Wei Dong, Li Qing, and Lei Chen. 2025. A Survey on Large Language Model Acceleration based on KV Cache Management. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=z3JZu9EA3>
- [39] Zihao Li. 2023. The dark side of chatgpt: Legal and ethical challenges from stochastic parrots and hallucination. *arXiv preprint arXiv:2304.14347* (2023).
- [40] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. <https://aclanthology.org/W04-1013/>
- [41] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 11 (2024), 157–173.
- [42] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [43] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems* 36 (2023), 43447–43478.

- [44] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Ruhle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 963–981. <https://aclanthology.org/2024.findings-acl.57>
- [45] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.
- [46] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.
- [47] Ritesh Sarkhel and Arnab Nandi. 2019. Visual segmentation for information extraction from heterogeneous visually rich documents. In *Proceedings of the 2019 international conference on management of data*. 247–262.
- [48] Ritesh Sarkhel and Arnab Nandi. 2021. Improving information extraction from visually rich documents using visual span representations. *Proceedings of the VLDB Endowment* 14, 5 (2021).
- [49] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592* (2025).
- [50] Jun Song, Jingyi Ding, Irshad Kandy, Yanghao Lin, Zhongjia Wei, Zilong Zhou, Zhiwei Peng, Jixi Shan, Hongyue Mao, Xiuqi Huang, et al. 2025. Magnus: A Holistic Approach to Data Management for Large-Scale Machine Learning Workloads. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4964–4977.
- [51] Alon Talmor, Ori Yoran, Amnon Catav, Dan Lahav, Yizhong Wang, Akari Asai, Gabriel Ilharco, Hannaneh Hajishirzi, and Jonathan Berant. 2021. Multi-modalqa: Complex question answering over text, tables and images. *International Conference on Learning Representations* (2021).
- [52] Jianheng Tang, Weiqi Zhang, Jiajin Li, Kangfei Zhao, Fugee Tsung, and Jia Li. 2023. Robust attributed graph alignment via joint structure learning and optimal transport. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 1638–1651.
- [53] Jianheng Tang, Xi Zhao, Lemin Kong, Xiaofang Zhou, and Jia Li. 2025. Fused Gromov-Wasserstein Alignment for Graph Edit Distance Computation and Beyond. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3641–3654.
- [54] Matthias Urban and Carsten Binnig. 2024. Eleet: Efficient learned query execution over text and tables. *Proceedings of the VLDB Endowment* 17, 13 (2024), 4867–4880.
- [55] Mengzhao Wang, Xiangyu Ke, Xiaoliang Xu, Lu Chen, Yunjun Gao, Pinpin Huang, and Runkai Zhu. 2024. Must: An effective and scalable framework for multimodal search of target modality. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4747–4759.
- [56] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, and Lu Chen. 2024. An Interactive Multi-Modal Query Answering System with Retrieval-Augmented Large Language Models. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4333–4336.
- [57] Yubo Wang, Haoyang Li, Fei Teng, and Lei Chen. 2026. AGRAG: Advanced Graph-based Retrieval-Augmented Generation for LLMs. *2026 IEEE 42th International Conference on Data Engineering (ICDE)* (2026).
- [58] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv:2309.07597 [cs.CL]*
- [59] Mingji Yang, Hanzhi Wang, Zhewei Wei, Sibao Wang, and Ji-Rong Wen. 2024. Efficient algorithms for personalized pagerank computation: A survey. *IEEE Transactions on Knowledge and Data Engineering* 36, 9 (2024), 4582–4602.
- [60] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.
- [61] Qinhan Yu, Zhiyou Xiao, Binghui Li, Zhengren Wang, Chong Chen, and Wentao Zhang. 2025. MRAMG-Bench: A Comprehensive Benchmark for Advancing Multimodal Retrieval-Augmented Multimodal Generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3616–3626.
- [62] Xu Yuan, Liangbo Ning, Wenqi Fan, and Qing Li. 2025. mKG-RAG: Multimodal Knowledge Graph-Enhanced RAG for Visual Question Answering. *arXiv preprint arXiv:2508.05318* (2025).
- [63] Haoyu Zhang, Jun Liu, Zhenhua Zhu, Shulin Zeng, Maojia Sheng, Tao Yang, Guohao Dai, and Yu Wang. 2024. Efficient and effective retrieval of dense-sparse hybrid vectors using graph-based approximate nearest neighbor search. *arXiv preprint arXiv:2410.20381* (2024).
- [64] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A survey of graph retrieval-augmented generation for customized large language models. *arXiv preprint arXiv:2501.13958* (2025).
- [65] Tao Zhang, Ziqi Zhang, Zongyang Ma, Yuxin Chen, Zhongang Qi, Chunfeng Yuan, Bing Li, Junfu Pu, Yuxuan Zhao, Zehua Xie, et al. 2024. mR<sup>2</sup>-AG: Multimodal Retrieval-Reflection-Augmented Generation for Knowledge-Based VQA. *arXiv preprint arXiv:2411.15041* (2024).
- [66] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren’s song in the AI ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219* (2023).
- [67] Zhijie Zhang, Yujie Lu, Weiguo Zheng, and Xuemin Lin. 2024. A comprehensive survey and experimental study of subgraph matching: trends, unbiasedness, and interaction. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–29.
- [68] Ruochen Zhao, Hailin Chen, Weishi Wang, Fangkai Jiao, Do Xuan Long, Chengwei Qin, Bosheng Ding, Xiaobao Guo, Minzhi Li, Xingxuan Li, et al. [n.d.]. Retrieving Multimodal Information for Augmented Generation: A Survey. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- [69] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [70] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, et al. 2025. In-depth Analysis of Graph-based RAG in a Unified Framework. *arXiv preprint arXiv:2503.04338* (2025).

**Table 12: Graph statistics of Graph-based RAG systems, where E. denotes entities and R. denotes relations. As MGRAG’s graph is query specific, its node and edge number denote the average statistic per query.**

| System   | MMQA |      | Wiki |      | Wit |      | Recipe |      | Arxiv |      | Manual |      | Web |     |
|----------|------|------|------|------|-----|------|--------|------|-------|------|--------|------|-----|-----|
|          | E.   | R.   | E.   | R.   | E.  | R.   | E.     | R.   | E.    | R.   | E.     | R.   | E.  | R.  |
| GraphRAG | -    | -    | 114  | 8    | 172 | 11   | 28     | 4    | 249   | 13   | 19     | 1    | 20  | 4   |
| LightRAG | -    | -    | 493  | 252  | 393 | 280  | 132    | 67   | 538   | 419  | 173    | 60   | 154 | 47  |
| HippoRAG | 620k | 11k  | 13k  | 5k   | 14k | 7k   | 20k    | 7k   | 14k   | 10k  | 5k     | 2k   | 10k | 3k  |
| MGRAG    | 8.6  | 11.2 | 7.9  | 10.2 | 9.9 | 11.5 | 14.8   | 20.3 | 10.9  | 14.6 | 13.1   | 16.5 | 7.3 | 9.3 |

## APPENDIX

### A THEOREMS AND PROOFS

#### A.1 Hardness of MCMI Generation

Next, we prove the *Path-based Semantic Subgraph Matching Problem* as NP-hard, and can be reduced from the NP-hard *Subgraph Isomorphism* problem [21, 67]:

**THEOREM 3.** *The Path-based Semantic Subgraph Matching Problem is NP-hard.*

**PROOF.** To demonstrate that the Path-based Semantic Subgraph Matching Problem is NP-hard, we provide a polynomial-time reduction from the Subgraph Isomorphism problem. Since Subgraph Isomorphism is NP-complete [21], it suffices to show that any instance of Subgraph Isomorphism can be transformed into an equivalent instance of our problem.

Given an instance of Subgraph Isomorphism with a query graph  $Q\mathcal{G}_q$  and a multimodal KG  $K\mathcal{G}_q$ , we construct an instance of the Path-based Semantic Subgraph Matching Problem as follows:

- (1) Assign identical embedding vectors to all nodes and edges in both  $Q\mathcal{G}_q$  and  $K\mathcal{G}_q$ , i.e.,  $\vec{h}_v = \mathbf{c}$  for all  $v \in \mathcal{V}_q \cup \mathcal{V}_q$ , and  $\vec{h}_{e(u,r,v)} = \mathbf{c}$  for all edges, where  $\mathbf{c}$  is a constant vector;
- (2) Set the similarity threshold  $\theta = 1 - \epsilon$  for an arbitrarily small  $\epsilon > 0$ ;

Under this construction, the cosine similarity between any two embeddings is exactly 1, which is greater than  $\theta$ . Consequently, condition (iii) in Definition 2 is always satisfied as long as the structural conditions (i) and (ii) hold. Therefore, a mapping  $I$  is a valid path-based subgraph isomorphism in our problem if and only if it is a valid subgraph isomorphism in the original problem. It follows that there exists a solution to the Subgraph Isomorphism instance if and only if there exists a matched subgraph in the constructed Path-based Semantic Subgraph Matching instance. Thus, we prove that the Path-based Semantic Subgraph Matching Problem is NP-hard.  $\square$

#### A.2 Effectiveness Guarantee

Next, we prove our Path-based Semantic Subgraph Matching algorithm has an approximation ratio bounded by the similarity threshold  $\theta$  via  $\infty$ -Wasserstein Bottleneck [9].

**THEOREM 4.** *The Algorithms 1 achieves an approximation ratio bounded by the similarity threshold  $\theta$  for the Path-based Semantic Subgraph Matching Problem, if a valid matched subgraph is found.*

**PROOF.** We analyze the approximation guarantee by framing the graph matching process as a constrained optimal transport problem. Let  $T : \hat{\mathcal{E}}_q \rightarrow \mathcal{P}(\mathcal{K}\mathcal{G}_q)$  be the mapping that assigns each query edge  $\hat{e}_{(u,v)}$  to a valid topological path  $P$  in the knowledge graph. First, we define the transport cost function  $c(e, P)$  in the cosine metric space as the semantic distance between the target joint embedding and the aggregated path embedding:

$$c(e, P) = 1 - \cos(\vec{h}_{(u,v)}, \vec{h}_P^{agg})$$

The overall objective of our problem (as defined in Definition 4) is to maximize the semantic score  $S(I)$ . Minimizing the average transport cost is exactly equivalent to maximizing  $S(I)$ , which corresponds to the 1-Wasserstein distance ( $W_1$ ) of the discrete mapping under a uniform distribution:

$$W_1(T) = \frac{1}{|\hat{\mathcal{E}}_q|} \sum_{e \in \hat{\mathcal{E}}_q} c(e, T(e)) = 1 - S(I)$$

According to the strict validation criterion in Algorithm 2, our greedy algorithm accepts a mapping  $I_{greedy}$  if and only if every single mapped path strictly satisfies the threshold constraint:

$$\cos(\vec{h}_{(u,v)}, \vec{h}_P^{agg}) > \theta.$$

In the context of optimal transport, this strict local constraint acts on the supremum of the point-wise cost, effectively bounding the  $\infty$ -Wasserstein distance (the bottleneck distance) of the accepted mapping  $I_{greedy}$ :

$$W_\infty(T_{greedy}) = \max_{e \in \hat{\mathcal{E}}_q} c(e, T_{greedy}(e)) < 1 - \theta$$

By the fundamental monotonicity property of  $p$ -Wasserstein distances on probability measures (i.e.,  $W_p \leq W_q$  for  $p \leq q$ ), the average transport cost  $W_1$  is rigorously bounded by the bottleneck cost  $W_\infty$ :

$$1 - S(I_{greedy}) = W_1(T_{greedy}) \leq W_\infty(T_{greedy}) < 1 - \theta$$

This inequality mathematically guarantees that the overall semantic score of our greedy solution strictly satisfies:

$$S(I_{greedy}) > \theta$$

Let  $S_{opt}$  be the objective score of the absolute optimal mapping  $I^*$ . Because cosine similarity is inherently bounded by 1, the theoretical maximum possible score is  $S_{opt} \leq 1$ . Therefore, we can establish the rigorous approximation ratio of our greedy algorithm as:

$$\frac{S(I_{greedy})}{S_{opt}} \geq \frac{\theta}{1} = \theta$$

Thus, the discrete greedy expansion serves as a bottleneck-constrained transport mapping, natively guaranteeing a  $\theta$ -approximation of the optimal solution without requiring exhaustive combinatorial enumeration.  $\square$

#### A.3 Complexity Analysis

**A.3.1 Time Complexity.** For the Step 1: Coarse-grain Ranking. Denote the multimodal chunk set as  $|C|$ , the chunk length as  $l_c$ , and the embedding dimension as  $d_{emb}$  the time complexity with transformer-based encoder would be  $O(|C|d_{emb}l_c^2)$ .

For the Step 2: Fine-grain Retrieval. Denote the node set of query graph and multimodal KG as  $\hat{\mathcal{V}}$  and  $\mathcal{V}$ , respectively, the

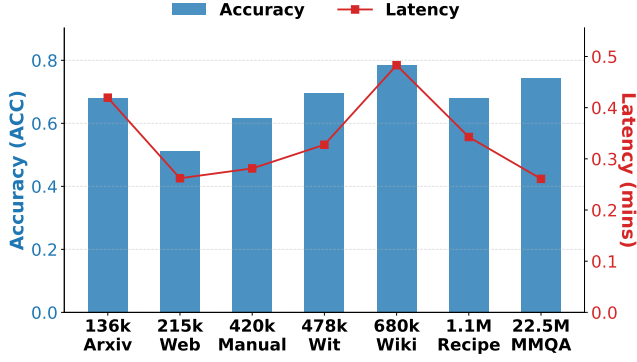


Figure 6: Accuracy and end-to-end latency of MGRAG on datasets with various corpus size.

number of edges in query graph as  $\hat{\mathcal{E}}$ , the maximum path context length as  $l_p$ , the query token length as  $l_q$ , the number of chunks used for graph construction as  $k$ , and the LLM’s hidden states as  $d_{LLM}$ . For each query, the entity and relation extraction would cost  $O(d_{emb}(kl_c^2 + l_q^2))$ , usually  $l_c \gg l_q$ , hence we can have the time complexity as  $O(d_{emb}kl_c^2)$ . In the worst case, the time complexity of each subgraph isomorphism calculation with Path-based VF2 algorithm would be:  $O(|\mathcal{V}|^{|\hat{\mathcal{V}}|})$  [13]. And the total Path-based Semantic Subgraph Matching algorithm have complexity:  $O(l_p|\hat{\mathcal{E}}||\mathcal{V}|^{|\hat{\mathcal{V}}|+1})$ .

For the Step 3: Graph-Aware KV Caching. Denote the number of nodes and edges in multimodal KG as  $\mathcal{V}$  and  $\mathcal{E}$ , the token length of the matched subgraph sequence and query as  $|S_{res}|$  and  $|q|$ . The time complexity computing degree centrality, betweenness centrality, eigenvector centrality and closeness centrality would be  $O(|\mathcal{V}| + |\mathcal{E}|)$ ,  $O(|\mathcal{V}||\mathcal{E}|)$ ,  $O(|\mathcal{V}|^2)$ , and  $O(|\mathcal{V}||\mathcal{E}|)$ , respectively. The time complexity computing cross attention score would be  $O(d_{LLM}(|S_{res}|^2 + |q|^2))$ .

Hence, the total offline time complexity of MGRAG would be:

$$O(|C|d_{emb}l_c^2),$$

and online time complexity would be:

$$O(d_{emb}kl_c^2 + l_p|\hat{\mathcal{E}}||\mathcal{V}|^{|\hat{\mathcal{V}}|+1} + |\mathcal{V}||\mathcal{E}| + d_{LLM}(|S_{res}|^2 + |q|^2)).$$

Although the time complexity is exponential in the worst case, according to Table 12, the number of nodes and edges in query graph and multimodal KG are less than 20 on average, MGRAG’s time cost is still comparable to other baselines.

**A.3.2 Space Complexity.** Denote the text and image documents as  $\mathcal{D}_{txt}$  and  $\mathcal{D}_{img}$ , respectively, the image captions as  $IC$ , the node and edge set of query graph and multimodal KG as  $\hat{\mathcal{V}}$ ,  $\mathcal{V}$ ,  $\hat{\mathcal{E}}$  and  $\mathcal{E}$ , respectively, the memory size of KV cache as  $m_c$ , the space complexity of MGRAG for each query would be:

$$O(|\mathcal{D}_{txt}| + |\mathcal{D}_{img}| + |\mathcal{E}| + |IC| + |\hat{\mathcal{V}}| + |\mathcal{V}| + |\hat{\mathcal{E}}| + |\mathcal{E}| + m_c)$$

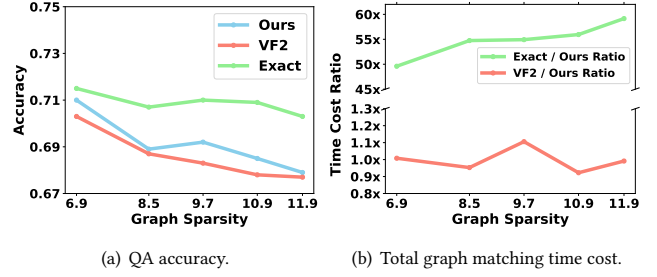


Figure 7: Accuracy and time cost comparison between our path-based semantic subgraph matching method and the exact solver VF2 [13] on graphs with varying densities sampled from MMQA dataset, where the x-axis denotes the graph density, y-axis denotes the QA accuracy and time cost.

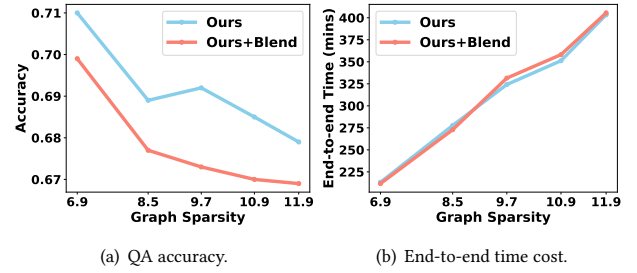


Figure 8: Accuracy and end-to-end time cost comparison of our MGRAG cope with our graph-based KV cache mechanism and CacheBlend (Blend), on graphs with varying sparsities sampled from MMQA dataset.

## B FURTHER EXPERIMENTS

### B.1 Performance across different corpus size

In Figure 6, we show the end-to-end latency and accuracy of MGRAG on datasets with varying corpus size. According to the Figure, although the MMQA has the largest corpus size (over 275k), it only costs the second most times than Recipe dataset, which has 2.5 times more queries than MMQA, indicating that the end-to-end latency of MGRAG mainly scales with the number of queries, but not the corpus size.

### B.2 Performance across different graph sparsity

We compare our graph-based KV cache mechanism with VF2 [13]-based exact solver and CacheBlend [60], on graphs with 5 different sparsity sampled from the MMQA dataset, in Figure 7 and Figure 8, respectively. Here, denote the number of graph nodes as  $|\mathcal{V}|$ , and the number of edges as  $|\mathcal{E}|$ , we define the graph sparsity as  $\frac{|\mathcal{V}|^2}{|\mathcal{E}|}$ , the higher this value is, the larger and sparser the graph is. According to the figure, we can see that our graph-based KV cache mechanism consistently outperforms the exact solver in efficiency and CacheBlend in effectiveness across different graph sparsity, demonstrating its better generalizability to varying graph sparsity.

### B.3 Performance on simple queries

According to the experiment results in Table 13. MGRAG only achieve comparable performance with VanillaRAG on ManyModalQA

Table 15: The inverse ablation of MGRAG’s path-based semantic subgraph matching (PSGM) and graph-based KV (GKV) cache mechanism, with two best performing RAG model HippoRAG2 [25] and VanillaRAG [20] on Manual.

| Model             | SIM.          | ACC.          | ROG.          | BLEU          | Time         |
|-------------------|---------------|---------------|---------------|---------------|--------------|
| Hippo.            | 0.6852        | 0.2641        | 0.2047        | 0.1314        | 2.4 h        |
| Hippo.+GKV        | 0.6808        | 0.2710        | 0.1998        | 0.1425        | 2.4 h        |
| Hippo.+PSGM       | 0.7958        | 0.5691        | 0.3741        | 0.2950        | 1.9 h        |
| Hippo.+PSGM+GKV   | <b>0.8221</b> | <b>0.6173</b> | <b>0.3839</b> | <b>0.3093</b> | 1.9 h        |
| Vanilla.          | 0.8166        | 0.5881        | 0.3509        | 0.2767        | <b>0.8 h</b> |
| Vanilla.+PSGM     | 0.8195        | 0.6003        | 0.3781        | 0.2975        | 2.0 h        |
| Vanilla.+PSGM+GKV | 0.8203        | 0.6095        | 0.3758        | 0.3010        | 2.0 h        |
| MGRAG             | <b>0.8221</b> | <b>0.6173</b> | <b>0.3839</b> | <b>0.3093</b> | <u>1.8 h</u> |

Table 13: The experiment result on ManyModalQA [27], which only consists of 1-hop queries, all experiments were carried out with Qwen2.5-VL-7B-Instruct backbone.

| Model      | SIM.          | ACC.          | ROG.          | BLEU.         | Time         |
|------------|---------------|---------------|---------------|---------------|--------------|
| VanillaRAG | <b>0.6672</b> | <b>0.5665</b> | <b>0.4766</b> | 0.4244        | <b>24.38</b> |
| HippoRAG2  | 0.6319        | 0.5097        | 0.4497        | 0.4131        | 54.40        |
| MGRAG      | <u>0.6585</u> | <u>0.5329</u> | <u>0.4749</u> | <b>0.4304</b> | <u>44.51</u> |

Table 14: Comparison between our graph-based KV cache mechanism and attention-based CacheBlend (C.Blend) [60] coping with our MGRAG pipeline on 6 datasets from MRANG [61] benchmark, the time is the end-to-end time in minutes.

| Method       | Metric | Wiki          | Wit           | Recipe        | Arxiv         | Manual        | Web           |
|--------------|--------|---------------|---------------|---------------|---------------|---------------|---------------|
| Ours + Blend | SIM.   | 0.8792        | 0.8011        | 0.8585        | 0.8651        | 0.8208        | 0.6772        |
|              | ACC.   | 0.7635        | 0.6908        | 0.6789        | 0.6263        | 0.6090        | 0.5035        |
|              | ROG.   | 0.3563        | 0.3559        | 0.4148        | 0.2616        | 0.3772        | 0.1651        |
|              | BLEU.  | 0.3018        | 0.2366        | 0.3019        | 0.2259        | 0.2992        | 0.1307        |
|              | Time   | 246.0         | <b>180.9</b>  | 816.2         | 84.1          | <b>109.3</b>  | 204.0         |
| Ours         | SIM.   | <b>0.8838</b> | <b>0.8033</b> | <b>0.8673</b> | <b>0.8760</b> | <b>0.8221</b> | <b>0.7009</b> |
|              | ACC.   | <b>0.7845</b> | <b>0.6967</b> | <b>0.6814</b> | <b>0.6800</b> | <b>0.6173</b> | <b>0.5123</b> |
|              | ROG.   | <b>0.3579</b> | <b>0.3589</b> | <b>0.4220</b> | <b>0.2726</b> | <b>0.3839</b> | <b>0.1664</b> |
|              | BLEU.  | <b>0.3174</b> | <b>0.2382</b> | <b>0.3102</b> | <b>0.2498</b> | <b>0.3093</b> | <b>0.1456</b> |
|              | Time   | <b>241.5</b>  | 196.5         | <b>808.7</b>  | <b>83.9</b>   | 109.7         | <b>196.5</b>  |

dataset [27], which only consists of simple 1-hop queries. This is because the graph-based retrieval and generation mechanism of MGRAG is designed to handle complex multi-hop reasoning scenarios, and may not provide significant advantages for simple queries that can be effectively addressed by traditional RAG methods. However, MGRAG still achieves competitive performance on this dataset, demonstrating its versatility across different query complexities.

#### B.4 Performance of KVCache mechanisms

We compare our graph-based KV cache mechanism with attention-based CacheBlend [60] coping with our MGRAG pipeline on 6 datasets from MRANG benchmark with the Qwen2.5-VL-7B-Instruct backbone. In the experiment, we fix other components of the MGRAG pipeline, and only change the KV cache mechanism to ensure a fair comparison. The result is shown in Table 14, where our graph-based KV cache mechanism consistently outperforms CacheBlend with comparable end-to-end time cost, demonstrating the effectiveness and efficiency of our graph-based KV cache mechanism.